Design Patterns

Pedro García López

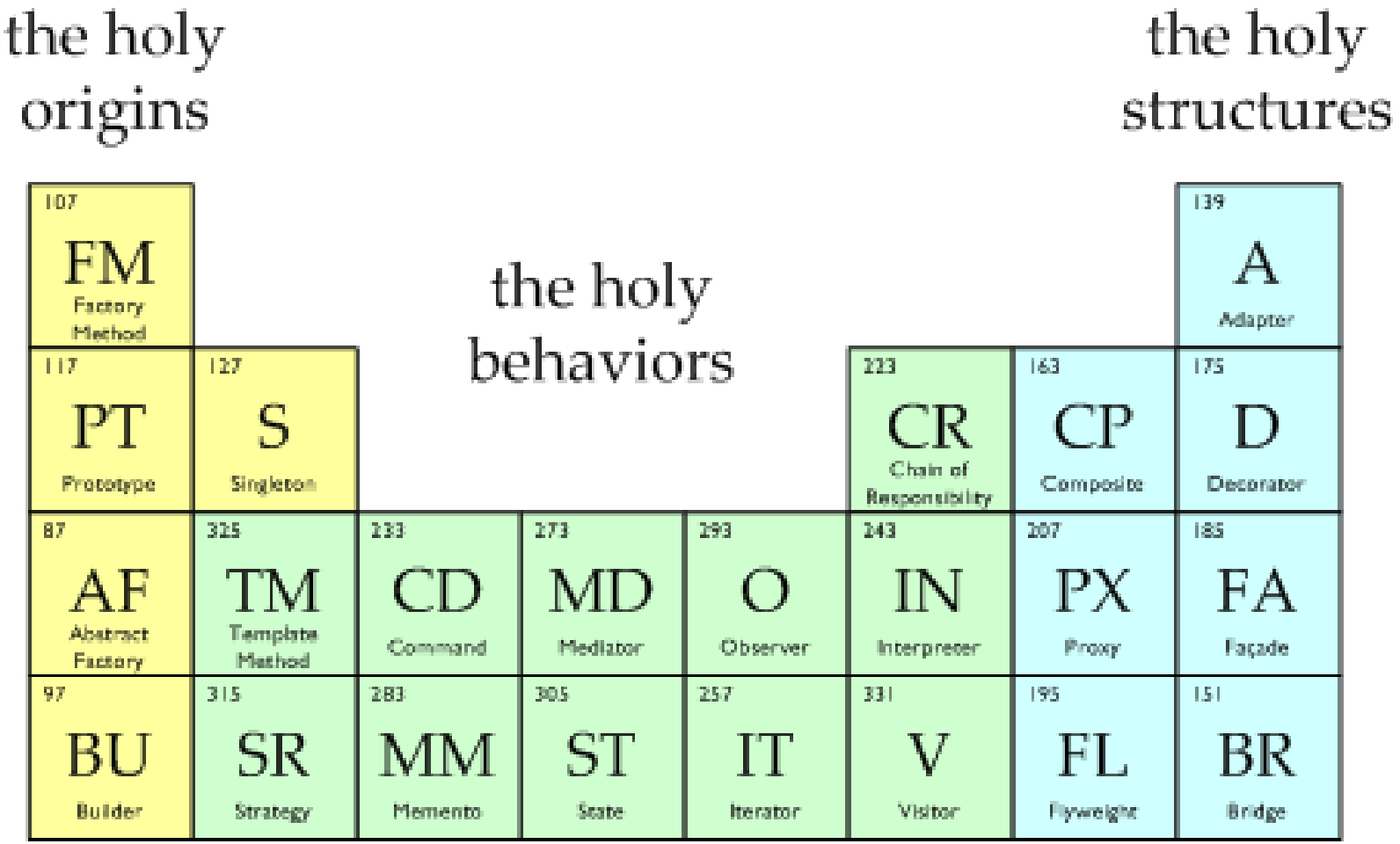
pgarcia@etse.urv.es/

Copyright

- © University Rovira i Virgili

- Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.1 or any later version published by the Free Software Foundation; provided its original author is mentioned and the link to http://libre.act-europe.fr/ is kept at the bottom of every non-title slide. A copy of the license is available at:
- http://www.fsf.org/licenses/fdl.html

The Sacred Elements of the Faith



Bibliography



Definition

- … a fully realized form, original, or model accepted or proposed for imitation**…[dictionary]**
- ... describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice **[Alexander]**
- … the abstraction from a concrete form which keeps recurring in specific non-arbitrary contexts **[Riehle]**

Birth of design patterns

- Q: experienced designers do make good design, but how?
- A: they reuse solutions that have worked for them in the past

- Then, how to capture design experience in a form that people can use effectively?
  - Gang of Four(Gamma, Helm, Johnson and Vlissides) collect and catalog them
  - Design Pattern are named after the work of Christopher Alexander, architect

Properties

Patterns do...

- provide common vocabulary
- provide “shorthand” for effectively communicating complex principles
- help document software architecture
- capture essential parts of a design in compact form
- show more than one solution
- describe software abstractions

Patterns do not...

- provide an exact solution
- solve all design problems
- only apply for object-oriented design

Description

Pattern

- **Context**
  - a design situation giving rise to a design problem
- **Problem**
  - a set of **forces** occuring in that context
- **Solution**
  - a form or rule that can be applied to **resolve** these forces

Example – window place

- forces
  - **he wants to sit down and be comfortable**
  - **he is drawn toward the light**
- solution
  - **in every room, make at least one window into a "window place"**

GoF Format

Consequences

trade-offs, concerns

Implementation

hints, techniques

Sample code

code fragment showing possible implementation

Known uses

patterns found in real systems

Related patterns

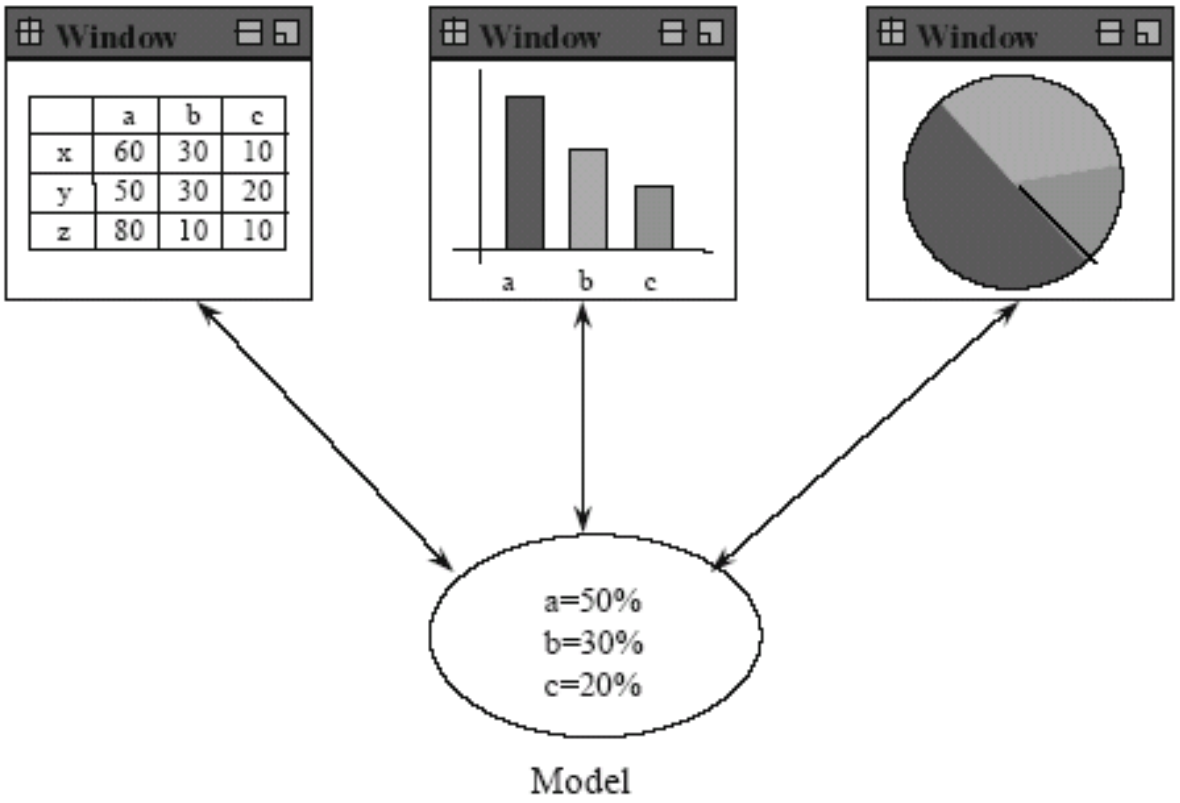
closely related patterns

Model View Controller (MVC)

- Used in Smalltalk to build user interfaces

- Model: Data structure or object, invisible
- View: Representation on the screen
- Controller: Encapsulates reactions on inputs of users, couples models and views

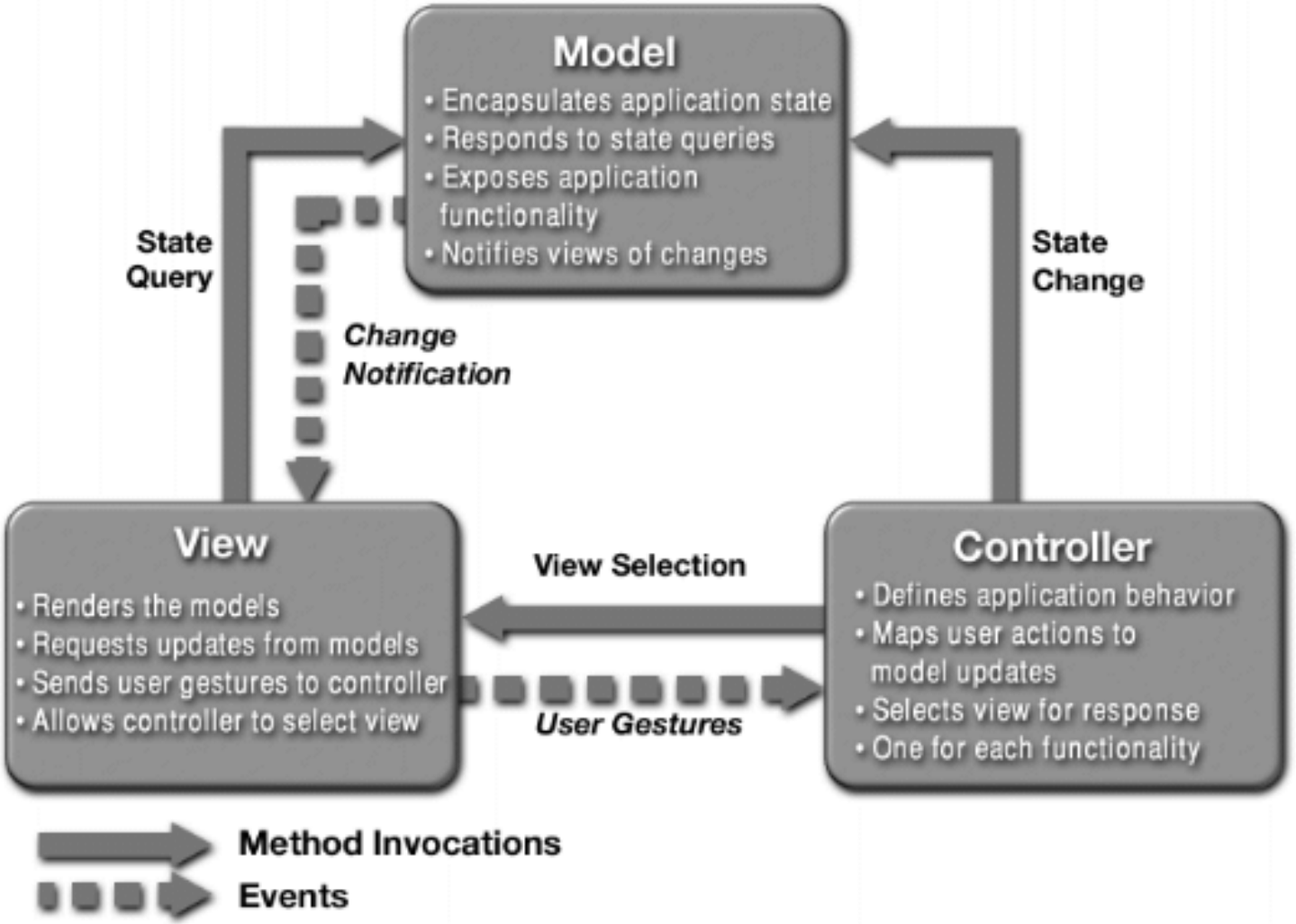
MVC


Window
Window
Window
a b c
x 60 30 10
y 50 30 20
z 80 10 10
a b c
a=50%
b=30%
c=20%
Model

MVC

- MVC decouples views and models by stablishing a subscribe/notify protocol between them.
- A view must ensure that its appearance reflects the state of the model
- Whenever the model's data changes, the model notifies views that depend on it.
- This approach lets you attach multiple views to a model to provide different representations

MVC


Model
• Encapsulates application state
• Responds to state queries
• Exposes application functionality
• Notifies views of changes
State Query
Change Notification
State Change
View
• Renders the models
• Requests updates from models
• Sends user gestures to controller
• Allows controller to select view
View Selection
User Gestures
Controller
• Defines application behavior
• Maps user actions to model updates
• Selects view for response
• One for each functionality
Method Invocations
Events

Pattern Catalog GoF

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	• **Factory Method**	• **Adapter**	• **Interpreter**
	Object	• **Abstract Factory** • **Builder** • **Prototype** • **Singleton**	• **Adapter** • **Bridge** • **Composite** • **Decorator** • **Facade** • **Flyweight** • **Proxy**	• **Chain of Responsibility** • **Command** • **Iterator** • **Mediator** • **Memento** • **Observer** • **State** • **Strategy** • **Vistor**

Design Patterns Benefits

- Design focusing in the interface instead of the implementation
- Facilitate Code Reuse
- Stress the importance of composition versus inheritance
- Designs are far more reusable using composition (fragile base class)

Inheritance vs composition

INHERITANCE

- static binding
- Easy to use
- Easy to change the implementation at compile time
- Break encapsulation

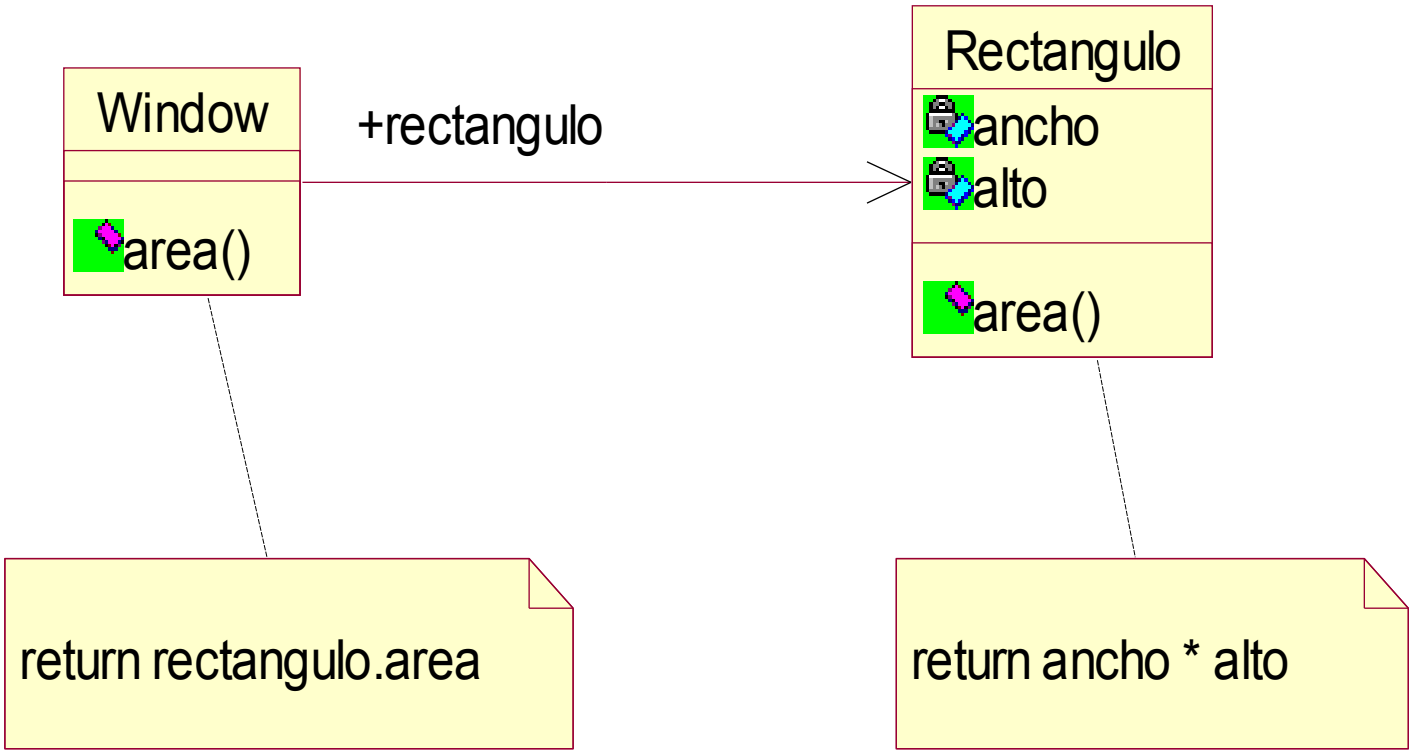
COMPOSITION

- dynamic binding
- More complicated
- Easy to change at runtime
- Does not break encapsulation
- Requires implementing interfaces

Delegation

- Delegation makes composition as powerful as inheritance
- The reveiver object delegates incoming calls to its delegate
- Delegation vs. *forwarding*
- More flexible Software & more complicated
- Used in several patterns: *State*, *Strategy*, *Visitor*,..
- Extreme case of composition. Shows that inheritance can be replaced with composition

Delegation


Window
area()
+rectangulo
Rectangulo
ancho
alto
area()
return rectangulo.area
return ancho * alto

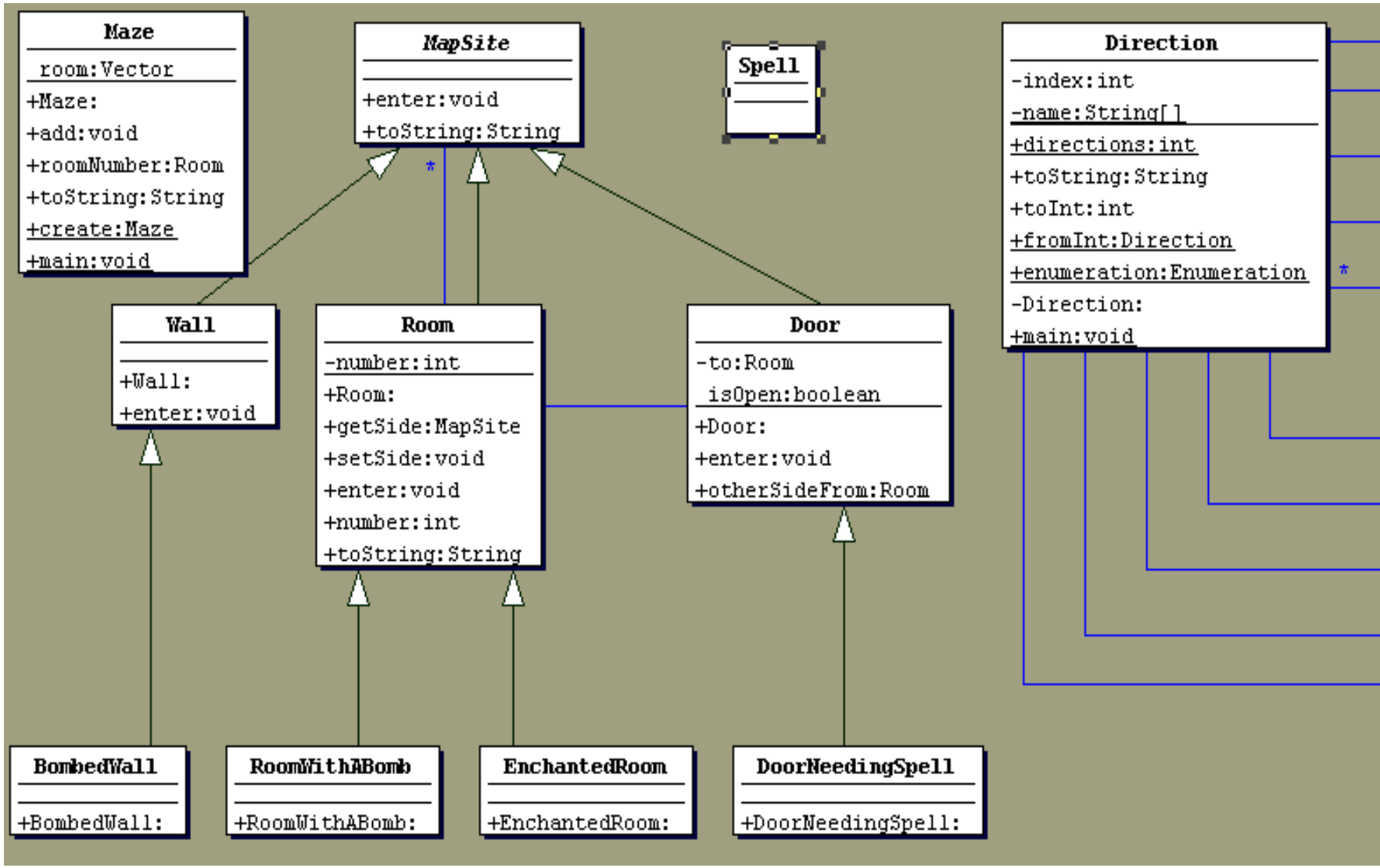
Creational Patterns

- Abstract the instantiation process.
  - A class creational pattern uses inheritance to vary the class that's instantiated
  - An object creational pattern will delegate instantiation to another object.
- Recurring Themes
  - Encapsulate knowledge about which concrete classes the system uses.
  - Hide how instances of these classes are created and put together.
- Consequences
  - Flexibility in
  - who creates it
  - what gets created
  - how it gets created
  - when it gets created.

Creational Patterns

- Abstract Factory
- Factory Method
- Builder
- Prototype
- Singleton

Example: Maze

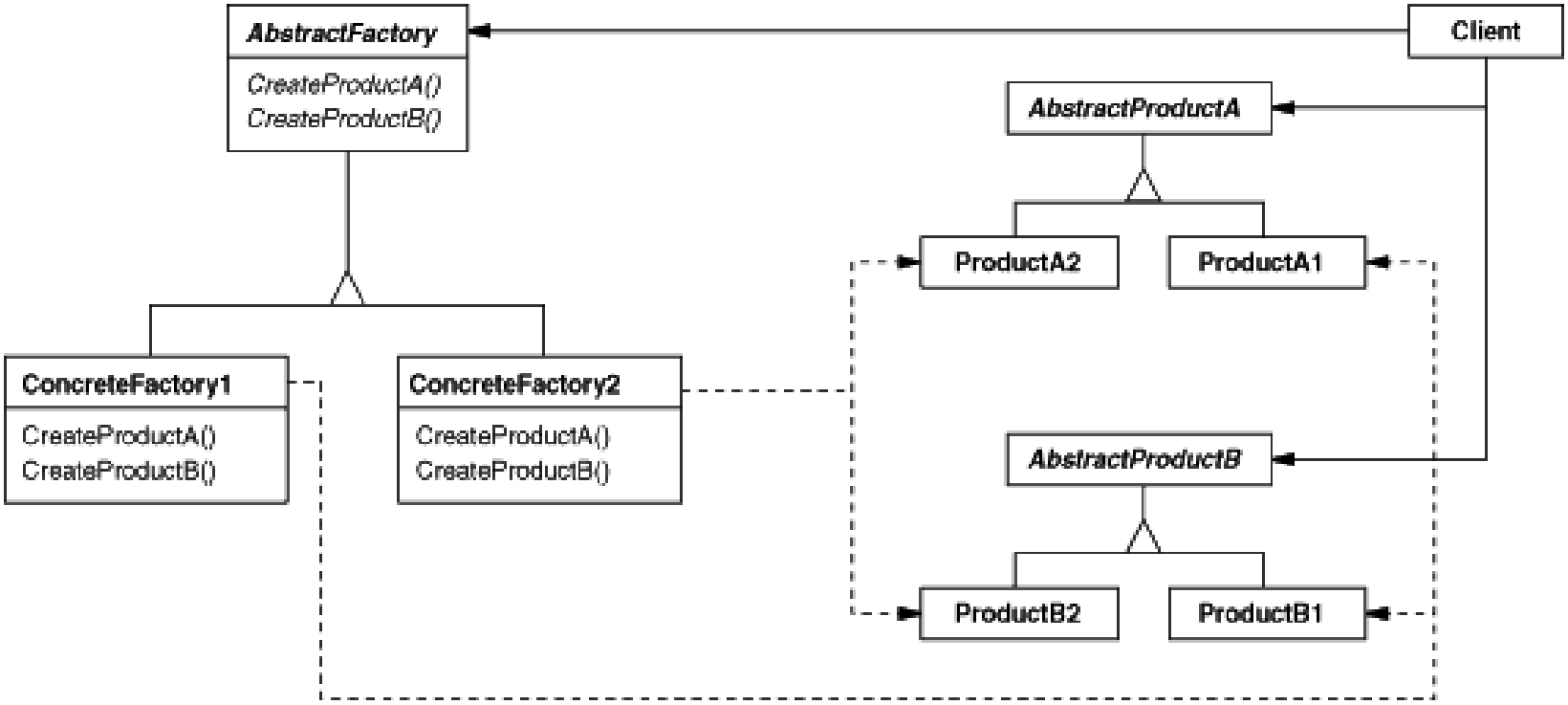


```
30 class Maze
31     {
32     public Maze() { }
33     public void add(Room room) { this.room.addElement(room); }
34     public Room roomNumber(int room)
35         {
36         for(int i=0;i<this.room.size();i++)
37             if(((Room)this.room.elementAt(i)).number()==room)
38                 return (Room)this.room.elementAt(i);
39         return null;
40         }
41     public String toString()
42         {
43         String string=new String();
44         for(int i=0;i<room.size();i++)
45             string+="\tRoom "+(i+1)+" "+(Room)room.elementAt(i);
46         return string;
47         }
48     public static Maze create()
49         {
50         Maze maze=new Maze();   // start creation
51         Room room1=new Room(1); // make a new rooom
52         Room room2=new Room(2); // make another room
53         Door door=new Door(room1, room2);   // add a door between them
54         maze.add(room1);        // add room1 to the maze
55         maze.add(room2);        // add room2 to the maze
56         room1.setSide(Direction.north,new Wall());
57         room1.setSide(Direction.east,door);
58         room1.setSide(Direction.south,new Wall());
59         room1.setSide(Direction.west,new Wall());
60         room2.setSide(Direction.north,new Wall());
61         room2.setSide(Direction.east,new Wall());
62         room2.setSide(Direction.south,new Wall());
63         room2.setSide(Direction.west,door);
64         return maze;
65         }
66     Vector room=new Vector();
67     public static void main(String[] arg)
68         {
69         Maze maze=Maze.create();
70         System.out.println(maze);
71         for(;;);
72         }
```

Example: Maze

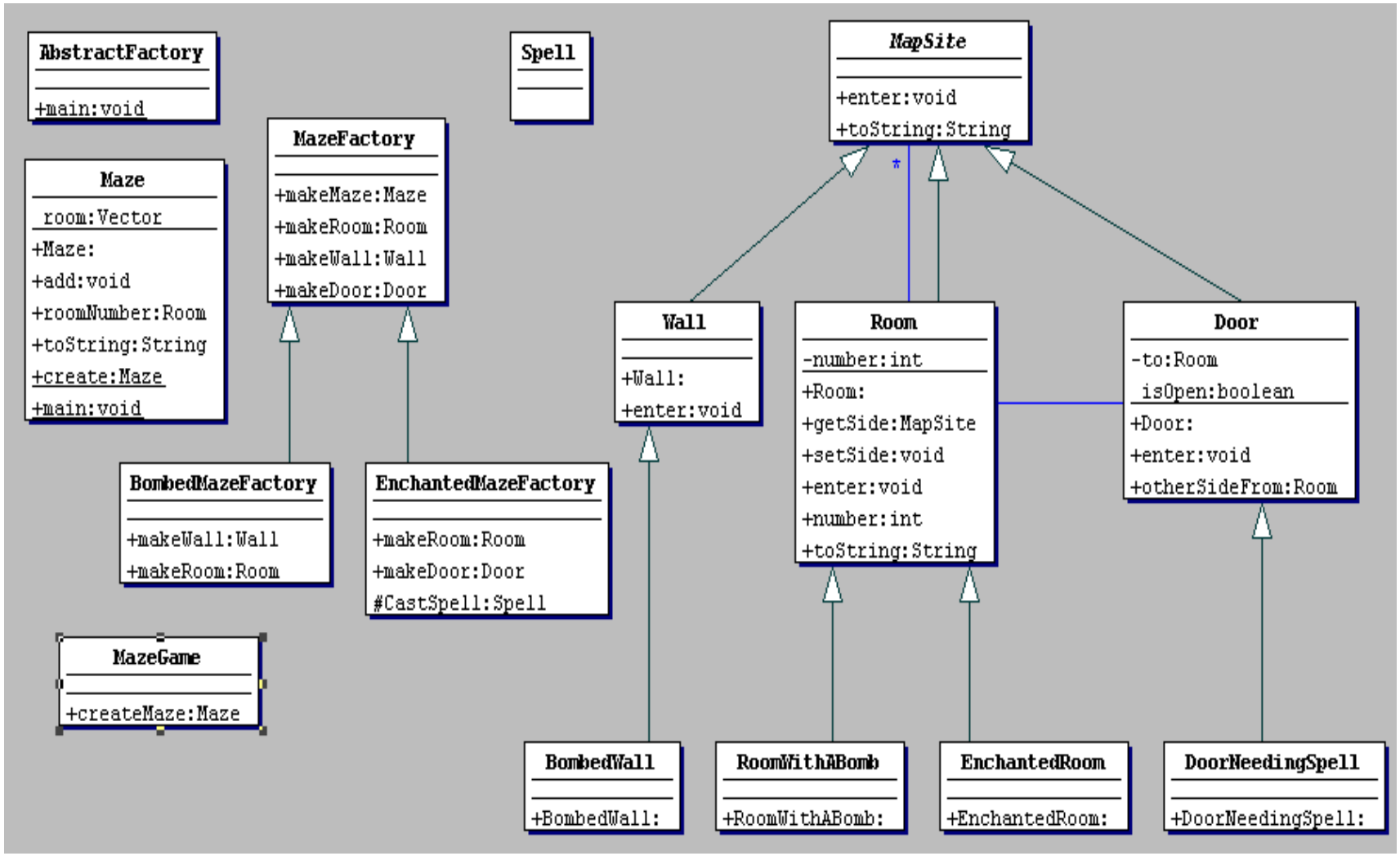
- The create() function is pretty complicated, considering that all it does is create a maze with two rooms.
- Inflexibility. It hard-codes the maze layout.
- How to create elements such as listeningDoors or MagicRooms ?
- Creational design patterns allow removal of explicit references to specific classes

Abstract factory



Provide an interface for creating families of related or dependent objects without specifying their concrete classes

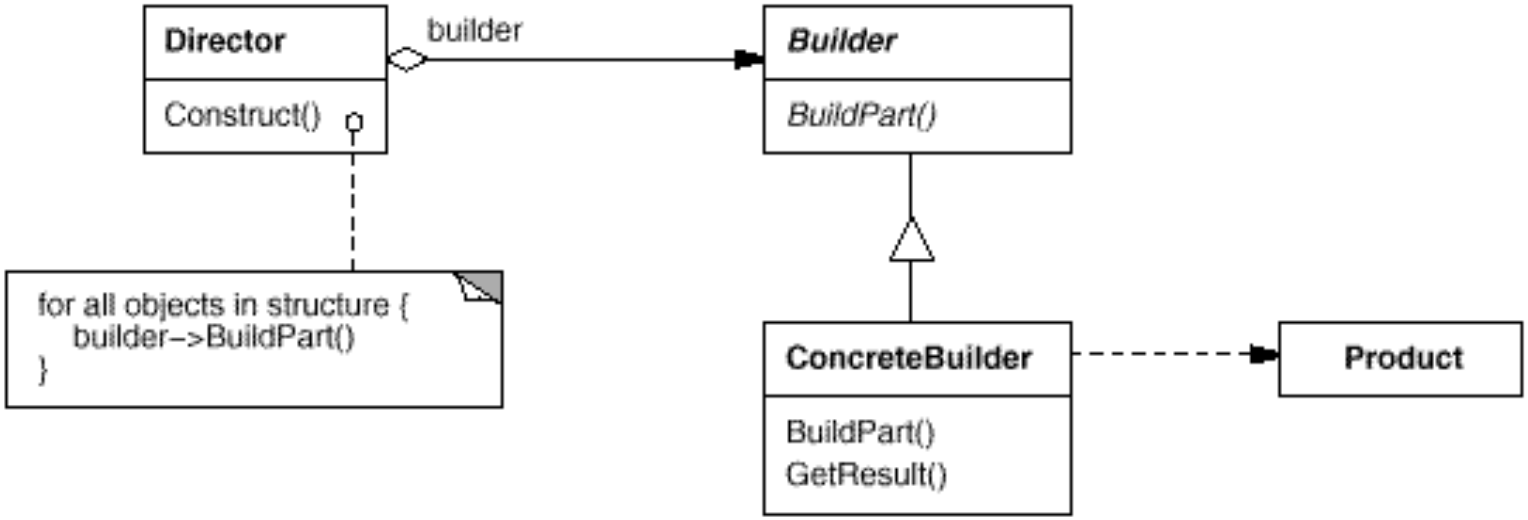
Abstract Factory



```
1  //If CreateMaze is passed an object as a parameter to use to create rooms, walls, and doors,
2  // then you can change the classes of rooms, walls, and doors by passing a different parameter.
3  import java.util.Vector;
4  class MazeFactory
5      {
6      public Maze makeMaze() { return new Maze(); }
7      public Room makeRoom(int number) { return new Room(number); }
8      public Wall makeWall() { return new Wall(); }
9      public Door makeDoor(Room room1,Room room2) { return new Door(room1,room2); }
10     }
11 class MazeGame
12     {
13     public Maze createMaze(MazeFactory factory)
14         { //object as a parameter to use to create rooms, walls, and doors,
15         System.out.println("creating Maze using "+factory.getClass().getName());
16         Maze maze=factory.makeMaze();
17         Room room1=factory.makeRoom(1);
18         Room room2=factory.makeRoom(2);
19         Door door=factory.makeDoor(room1,room2);
20         maze.add(room1);
21         maze.add(room2);
22         room1.setSide(Direction.north,factory.makeWall());
23         room1.setSide(Direction.south,factory.makeWall());
24         room1.setSide(Direction.east,door);
25         room1.setSide(Direction.west,factory.makeWall());
26         room2.setSide(Direction.north,factory.makeWall());
27         room2.setSide(Direction.south,factory.makeWall());
28         room2.setSide(Direction.east,factory.makeWall());
29         room2.setSide(Direction.west,door);
30         return maze;
31         }
32     }
33 /** @stereotype role*/
34 class BombedMazeFactory extends MazeFactory
35     {
36 /** @stereotype Foo*/
37     public Wall makeWall() { return new BombedWall(); }
38     public Room makeRoom(int n) { return new RoomWithABomb(n); }
39     }
40 class EnchantedMazeFactory extends MazeFactory
41     {
42     public Room makeRoom(int n) { return new EnchantedRoom(n,CastSpell()); }
43     public Door makeDoor(Room r1,Room r2) { return new DoorNeedingSpell(r1,r2); }
44     protected Spell CastSpell() { return new Spell(); }
45     }
```

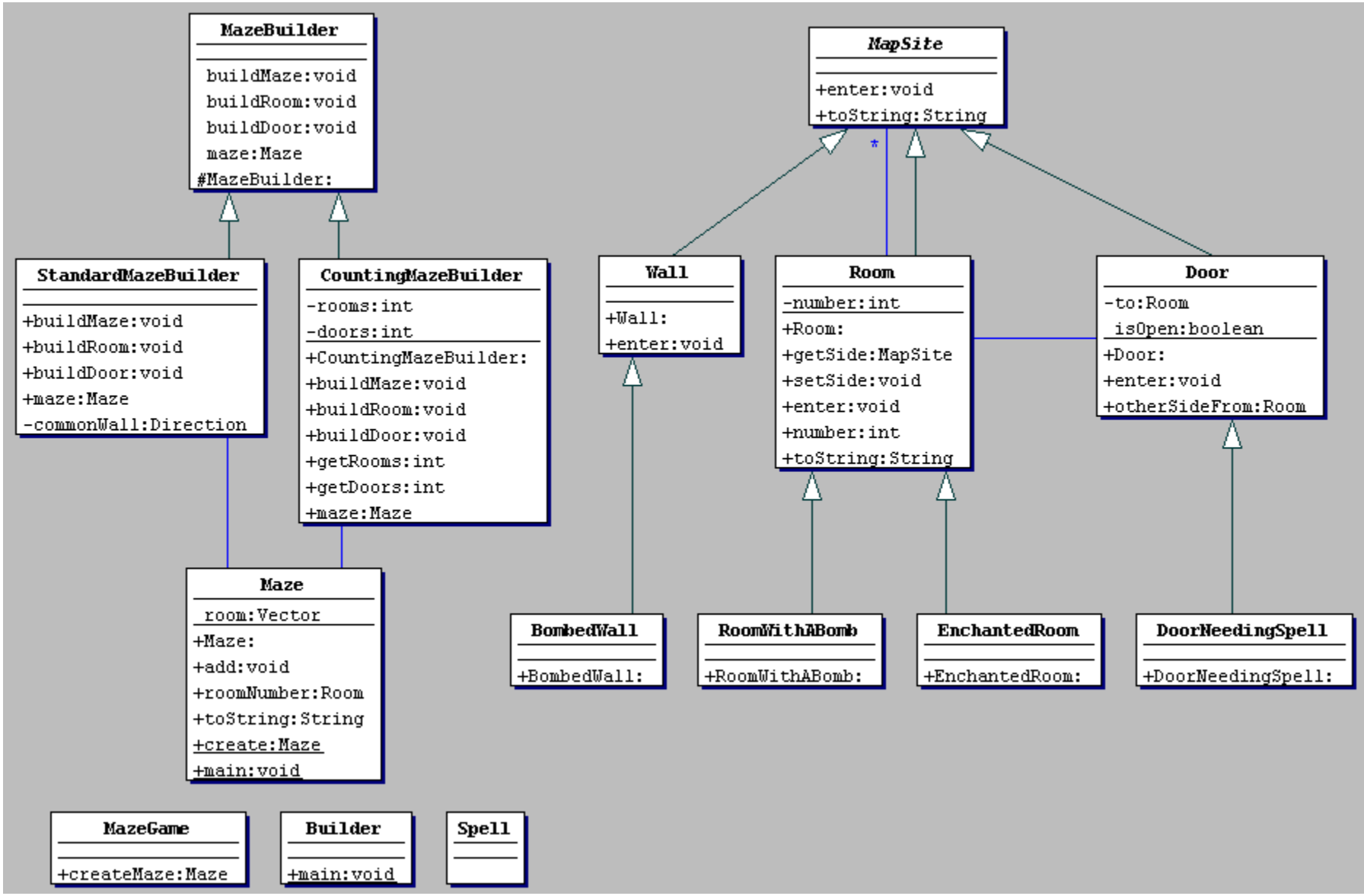
```
46 public class AbstractFactory
47     {
48     public static void main(String[] args)
49         {
50         Maze maze;
51         MazeGame game;
52         game=new MazeGame();
53         // make game with standard parts
54         maze=game.createMaze(new MazeFactory());
55         System.out.println(maze);
56         // change the classes of rooms, walls, and doors by passing a different parameter
57         maze=game.createMaze(new BombedMazeFactory());
58         System.out.println(maze);
59         game=new MazeGame();
60         // change the classes of rooms, walls, and doors by passing a different parameter
61         maze=game.createMaze(new EnchantedMazeFactory());
62         System.out.println(maze);
63         while(true)
64             ;
65         }
66     }
67
```

Builder



Separate the construction of complex object from its representation so that the same construction process can create different representations

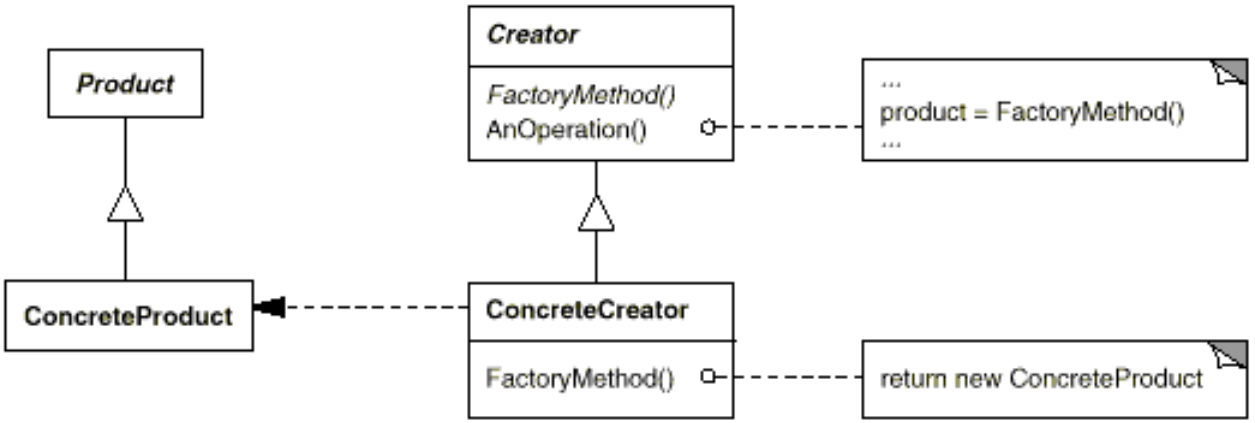
Builder


MazeBuilder
buildMaze:void
buildRoom:void
buildDoor:void
maze:Maze
#MazeBuilder:
MapSite
+enter:void
+toString:String
*
StandardMazeBuilder
+buildMaze:void
+buildRoom:void
+buildDoor:void
+maze:Maze
-commonWall:Direction
CountingMazeBuilder
-rooms:int
-doors:int
+CountingMazeBuilder:
+buildMaze:void
+buildRoom:void
+buildDoor:void
+getRooms:int
+getDoors:int
+maze:Maze
Wall
+Wall:
+enter:void
Room
-number:int
+Room:
+getSide:MapSite
+setSide:void
+enter:void
+number:int
+toString:String
Door
-to:Room
isOpen:boolean
+Door:
+enter:void
+otherSideFrom:Room
Maze
room:Vector
+Maze:
+add:void
+roomNumber:Room
+toString:String
+create:Maze
+main:void
BombedWall
+BombedWall:
RoomWithABomb
+RoomWithABomb:
EnchantedRoom
+EnchantedRoom:
DoorNeedingSpell
+DoorNeedingSpell:
MazeGame
+createMaze:Maze
Builder
+main:void
Spell

```
// If CreateMaze is passed an object that can create a new maze in its entirety using op
// then you can use inheritance to change parts of the maze or the way the maze is built
class MazeBuilder
    {
    void buildMaze() { }
    void buildRoom(int n) { }
    void buildDoor(int from,int to) { }
    Maze maze() { return null; }
    protected MazeBuilder() { };
    }
class StandardMazeBuilder extends MazeBuilder
    {
    public void buildMaze() { maze=new Maze(); }
    public void buildRoom(int n)
        {
        if(maze.roomNumber(n)==null)
            {
            Room r=new Room(n);
            maze.add(r);
            r.setSide(Direction.north,new Wall());
            r.setSide(Direction.south,new Wall());
            r.setSide(Direction.east,new Wall());
            r.setSide(Direction.west,new Wall());
            }
        }
    public void buildDoor(int from,int to)
        {
        Room r1=maze.roomNumber(from);
        Room r2=maze.roomNumber(to);
        Door d=new Door(r1,r2);
        r1.setSide(commonWall(r1,r2),d);
        r2.setSide(commonWall(r2,r1),d);
        }
    public Maze maze() { return maze; }
    private Direction commonWall(Room r1,Room r2) {return Direction.north;}
    private Maze maze;
    }
class CountingMazeBuilder extends MazeBuilder
    { // use inheritance to change parts of the maze or the way the maze is built.
    public CountingMazeBuilder() { rooms=doors=0; }
    public void buildMaze() { }
    public void buildRoom(int n) { rooms++; }
    public void buildDoor(int from,int to) { doors++; }
    public int getRooms() { return rooms; }
    public int getDoors() { return doors; }
    public Maze maze() { return maze; }
    private int rooms,doors;
    private Maze maze;
    }
```

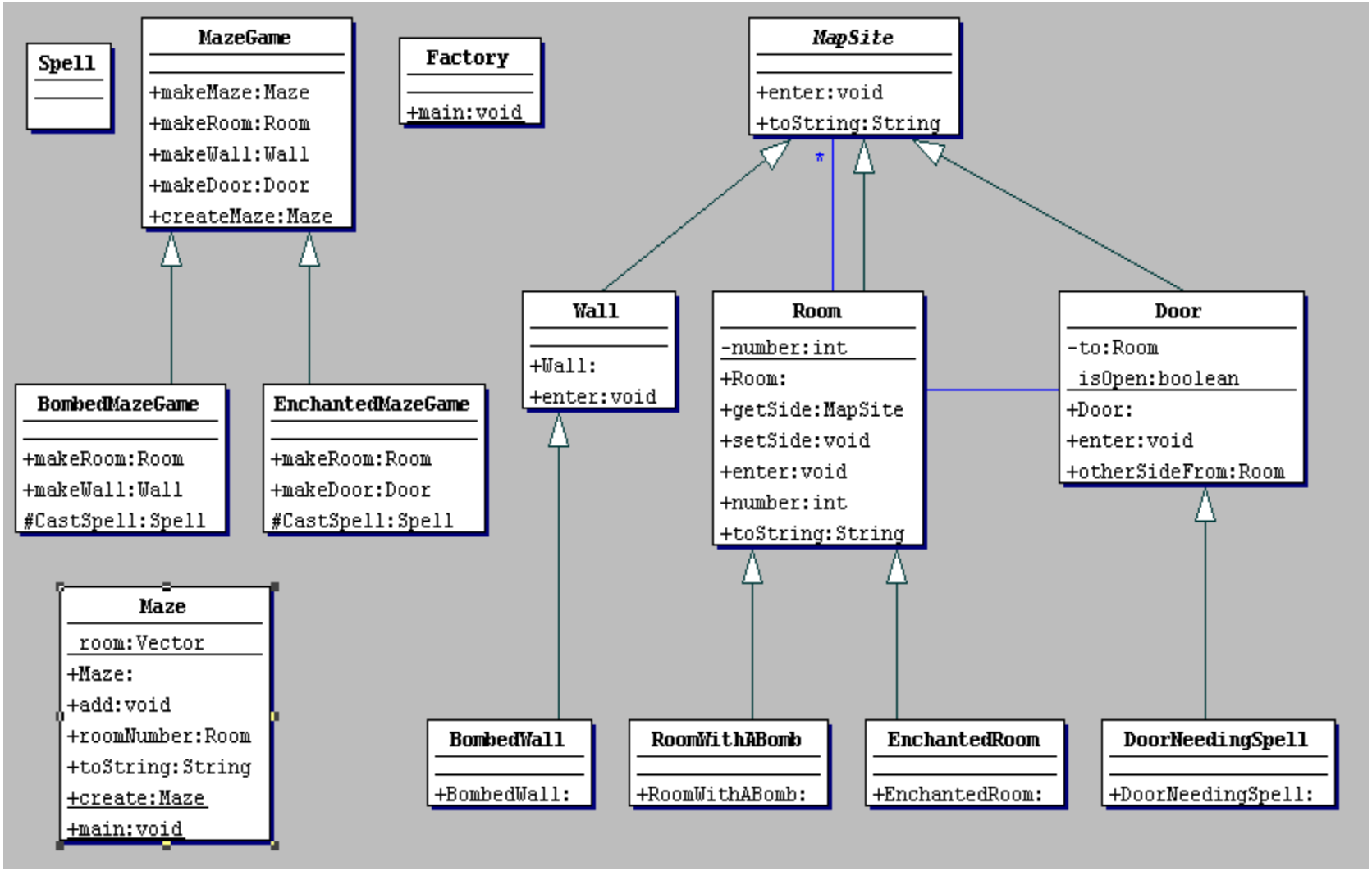
```
50 class MazeGame
51     {
52     public Maze createMaze(MazeBuilder builder)
53         {
54         builder.buildMaze();
55         builder.buildRoom(1);
56         builder.buildRoom(2);
57         builder.buildDoor(1,2);
58         return builder.maze();
59         }
60     }
61 public class Builder
62     {
63     public static void main(String arg[])
64         {
65         System.out.println("TestBulder");
66         Maze maze;
67         MazeGame game;
68         game=new MazeGame();
69         StandardMazeBuilder builder;
70         builder=new StandardMazeBuilder();
71         game.createMaze(builder);
72         System.out.println(builder.maze());
73         maze=builder.maze();
74         CountingMazeBuilder builder2;
75         builder2=new CountingMazeBuilder();
76         MazeGame game2=new MazeGame();
77         // If CreateMaze is passed an object that can create
78         game2.createMaze(builder2);
79         System.out.println("Rooms="+builder2.getRooms());
80         System.out.println("Doors="+builder2.getDoors());
81         for(;;);
82         }
83     }
84
```

Factory Method



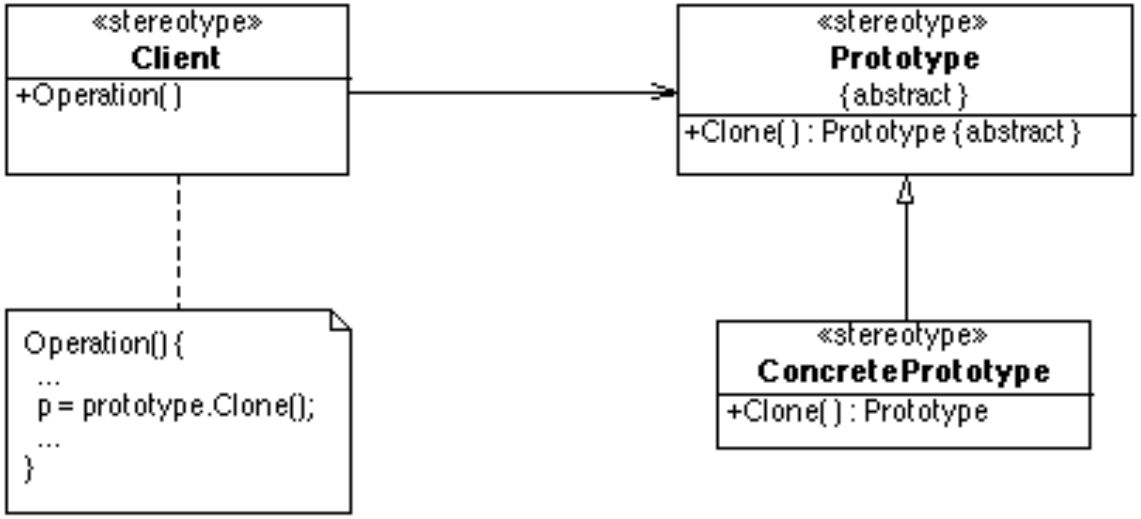
Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory method lets a class defer instantiation to subclasses

Factory Method



```
import java.util.*;
class MazeGame
    {
    public Maze makeMaze() { return new Maze(); }
    public Room makeRoom(int number) { return new Room(number); }
    public Wall makeWall() { return new Wall(); }
    public Door makeDoor(Room room1,Room room2) { return new Door(room1,room2); }
    public Maze createMaze()
        { // call virtual functions instead of constructor calls
        Maze maze=makeMaze();
        Room room1=makeRoom(1);
        Room room2=makeRoom(2);
        Door door=makeDoor(room1,room2);
        maze.add(room1);
        maze.add(room2);
        room1.setSide(Direction.north,makeWall());
        room1.setSide(Direction.east,door);
        room1.setSide(Direction.south,makeWall());
        room1.setSide(Direction.west,makeWall());
        room2.setSide(Direction.north,makeWall());
        room2.setSide(Direction.east,makeWall());
        room2.setSide(Direction.south,makeWall());
        room2.setSide(Direction.west,door);
        return maze;
        }
    }
class BombedMazeGame extends MazeGame
    { //subclass of MazeGame and redefining those virtual functions.
    public Room makeRoom(int n) { return new RoomWithABomb(n); }
    public Wall makeWall() { return new BombedWall(); }
    protected Spell CastSpell() { return new Spell(); }
    }
class EnchantedMazeGame extends MazeGame
    { //subclass of MazeGame and redefining those virtual functions.
    public Room makeRoom(int n) { return new EnchantedRoom(n,CastSpell()); }
    public Door makeDoor(Room room1,Room room2) { return new DoorNeedingSpell(room1,room2); }
    protected Spell CastSpell() { return new Spell(); }
    }
public class Factory
    {
    public static void main(String arg[])
        {
        System.out.println("TestFactory");
        Maze maze=null;
        MazeGame game;
        game=new BombedMazeGame();
        maze=game.createMaze();
        System.out.println(maze);
        game=new EnchantedMazeGame();
        maze=game.createMaze();
        System.out.println(maze);
        for(;;);
        }
    }
```

Prototype



Specifies the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.

Maze sample

```
public class MazePrototype {
      MazePrototype (Maze l, Door p, Room h,
            Wall m);
         {protoLab = l; protoPuerta = p; protoHab = h; protoMuro = m;}

     public Maze makeMaze() {return (Laberinto)protoLab.clone()};
     public Wall makeWall {return (Muro) protoMuro.clone() };
     public Room  makeRoom(int n) {
               Room h = (Room) protoHab.clone();
               h.initialize(n); return h;}
     public Door makeDoor (Room h1, Room h2)
           {Door p = (Door) protoPuerta.clone();
           p.initialize(h1,h2); return p}
}
```

```
Game juego;
 MazePrototype LaberintoSimple =
     new MazePrototype (new Laberinto, new Puerta,
                        new Habitacion, new Muro);
Maze unlab = juego. makeMaze(LaberintoSimple)
```

```
Game juego;
 MazePrototype otroLaberinto =
        new MazePrototype (new Laberinto, new
          PuertaQueOye, new HabitacionEncantada, new Muro);
Maze unlab = juego. makeMaze(otroLaberinto)
```

Database Sample (JDBC)

```
Connection conn = _ds.getConnection();

try {
  String sql = "SELECT name, cost FROM jdbc_basic_brooms ORDER BY cost DESC";

  Statement stmt = conn.createStatement();

  ResultSet rs = stmt.executeQuery(sql);

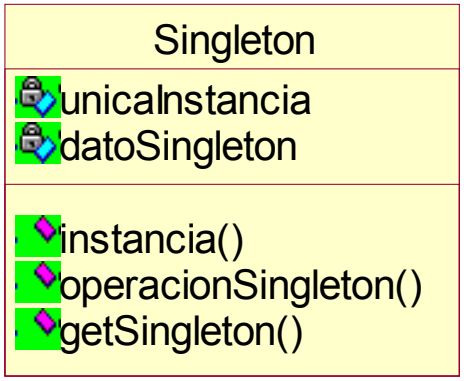
  out.println("<table border='3'>");

  while (rs.next()) {
    out.println("<tr><td>" + rs.getString(1));
    out.println("    <td>" + rs.getString(2));
  }

  out.println("</table>");

  rs.close();
  stmt.close();
} finally {
  conn.close();
}
```

Singleton



return unicaInstancia

```
public class Singleton {
        public static Singleton instancia()
        {    if (unicaInstancia = = null)
                {unicaInstancia = new Singleton()};
                return unicaInstancia
        };
        protected Singleton();
        private static Singleton unicaInstancia = null;
```

Ensure a class only has one instance, and provide a global point to access it.

Structural Patterns

- Structural patterns describe how classes and objects are combined to form larger structures
- Class patterns describe how inheritance can be used to provide more useful program interfaces
- Object patterns describe how objects can be composed into larger structures using object composition, or the inclusion of objects within objects

Structural Patterns

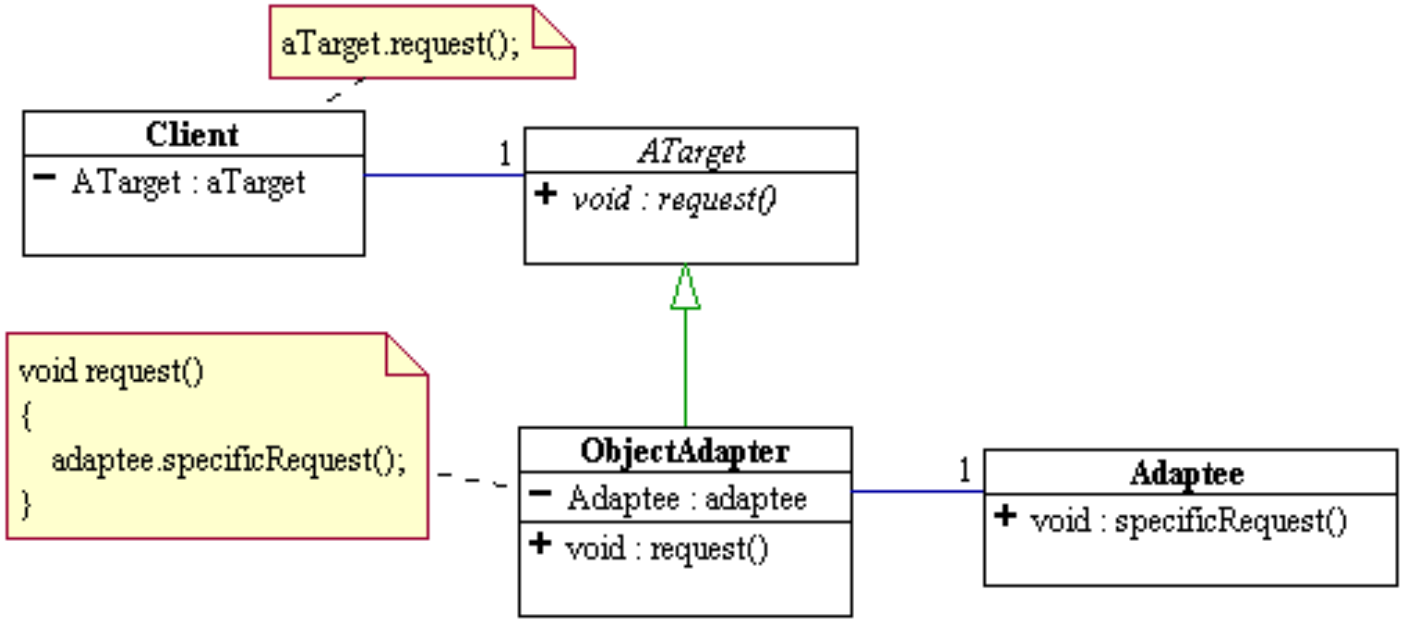
- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- Proxy

Adapter

- Convert the interface of a class into another interface clients expect. Adapter lets classes work together that could not otherwise because of incompatible interfaces.

Object Adapter

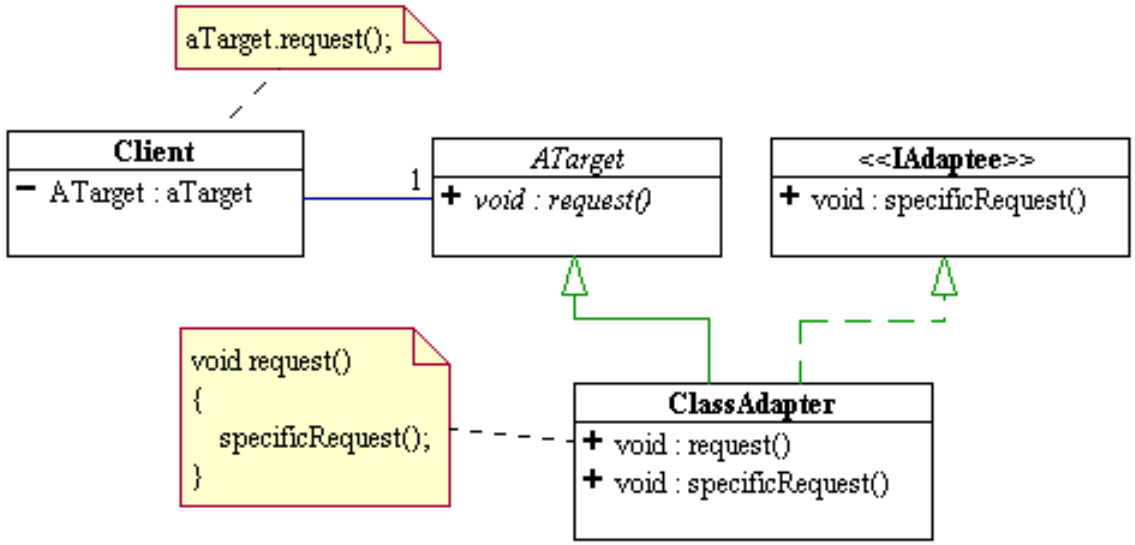
- The adapter inherits the target interface that the client expects to see, while it holds an instance the adaptee. When the client calls the request() method on its target object (the adapter), the request is translated into the corresponding specific request on the adaptee.



Class Adapter

- As in the object adapter, the class adapter inherits the interface of the client's target. However, it also inherits the interface of the adaptee as well.

  The request to the target is simply rerouted to the specific request that was inherited fro the adaptee interface.



```
interface Stack
{
  public void push (Object);
  public Object pop ();
  public Object top ();
}

/* DoubleLinkedList */
class DList
{
  public void insert (DNode pos, Object o);
  public void remove (DNode pos, Object o);

  public void insertHead (Object o);
  public void insertTail (Object o);

  public Object removeHead ();
  public Object removeTail ();

  public Object getHead ();
  public Object getTail ();
}

/* Adapt DList class to Stack interface */
class DListImpStack extends DList implements Stack
{
  public void push (Object o) {
    insertTail (o);
  }

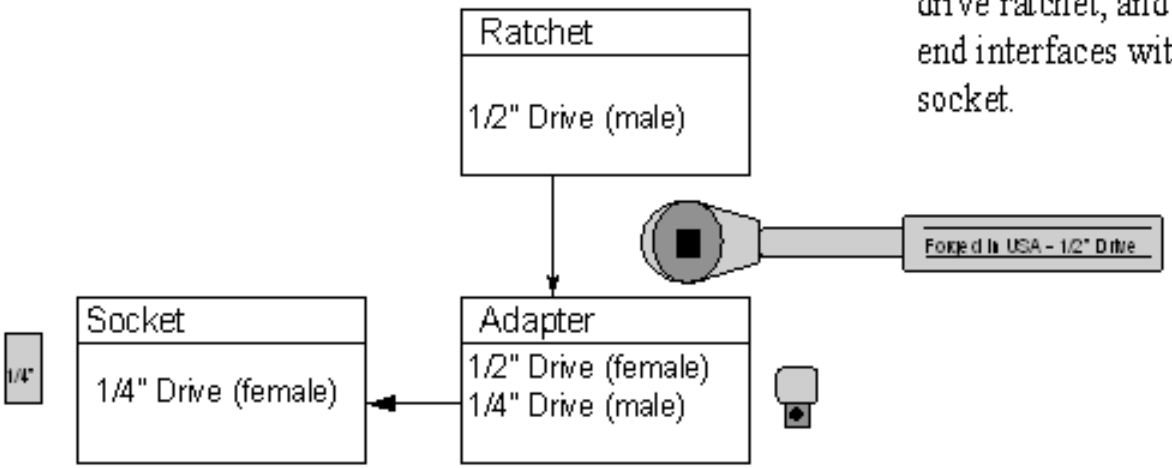
  public Object pop () {
    return removeTail ();
  }

  public Object top () {
    return getTail ();
  }
};
```

Adapter sample

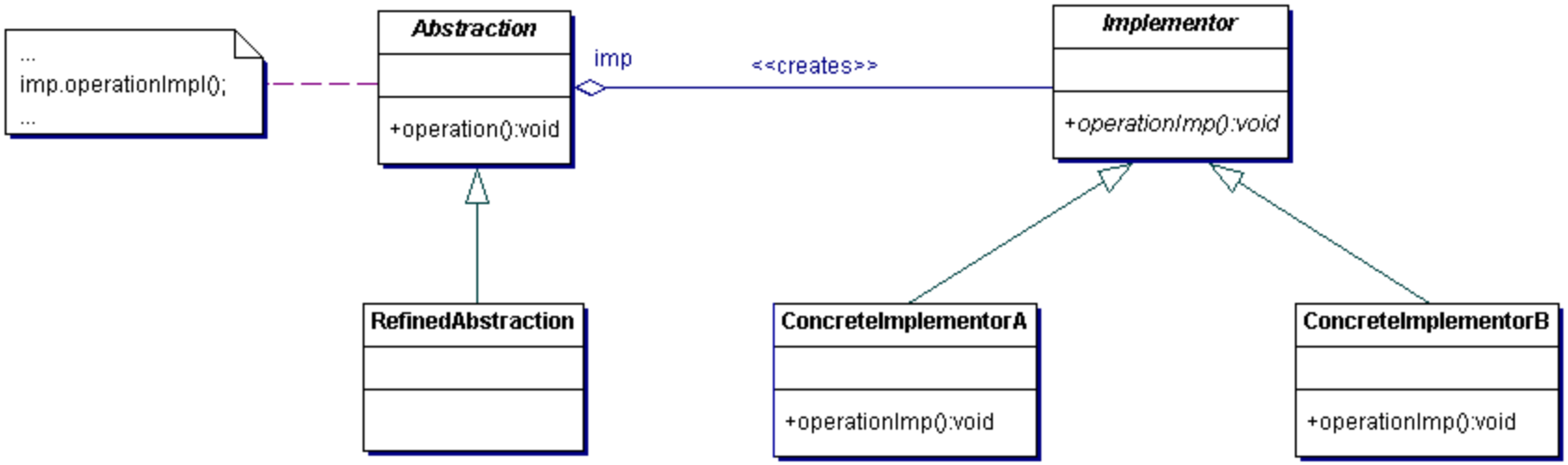
Adapter

A 1/2" drive ratchet will not ordinarily work with a 1/4" drive socket. Using an *Adapter*, the female end interfaces with the 1/2" drive ratchet, and the male end interfaces with the 1/4" socket.



Bridge

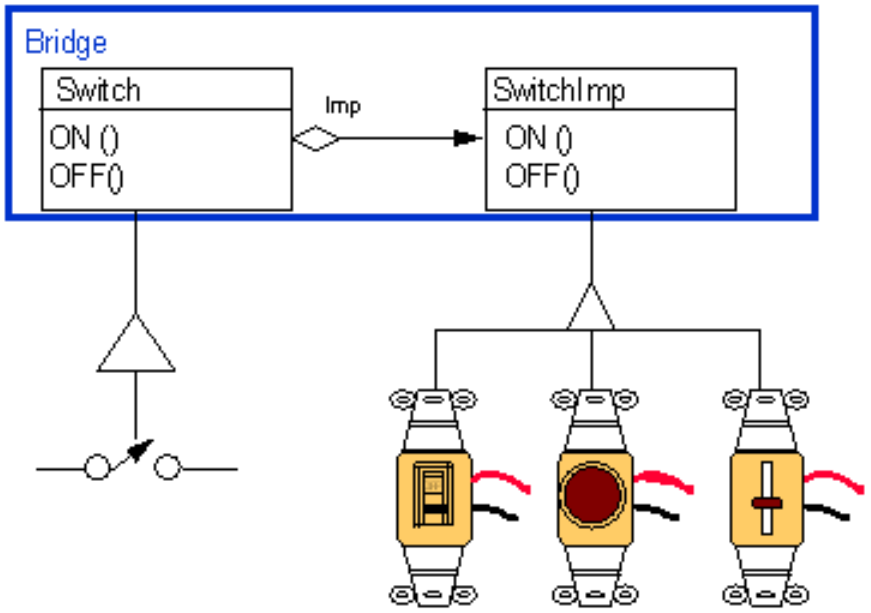
- Decouple an abstraction from its implementation so that the two can vary independently



Bridge sample

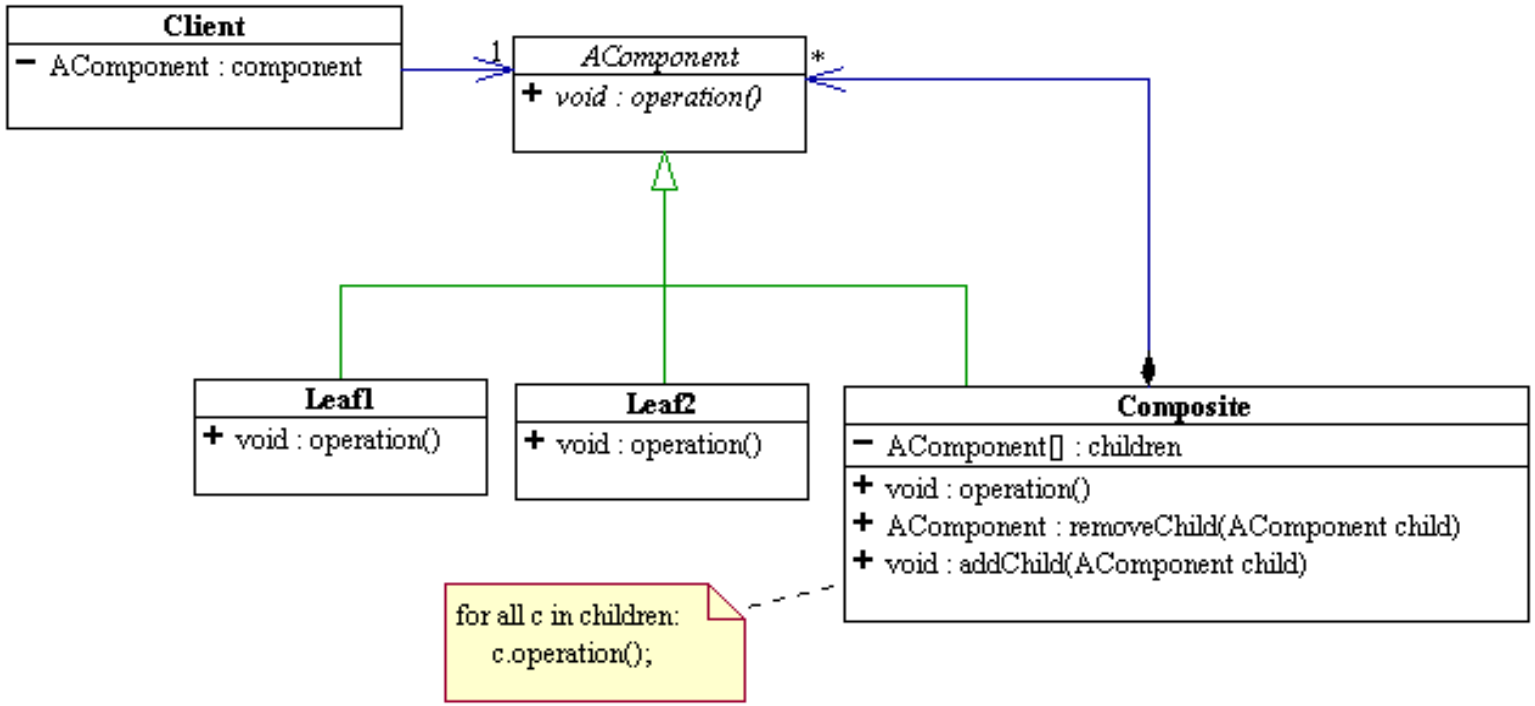
Bridge

A switch is a device for turning lights, ceiling fans, garbage disposals on or off. The actual implementation of the switch is decoupled from the abstract switch. This decoupling of the abstraction and implementation is an example of the *Bridge* Pattern.

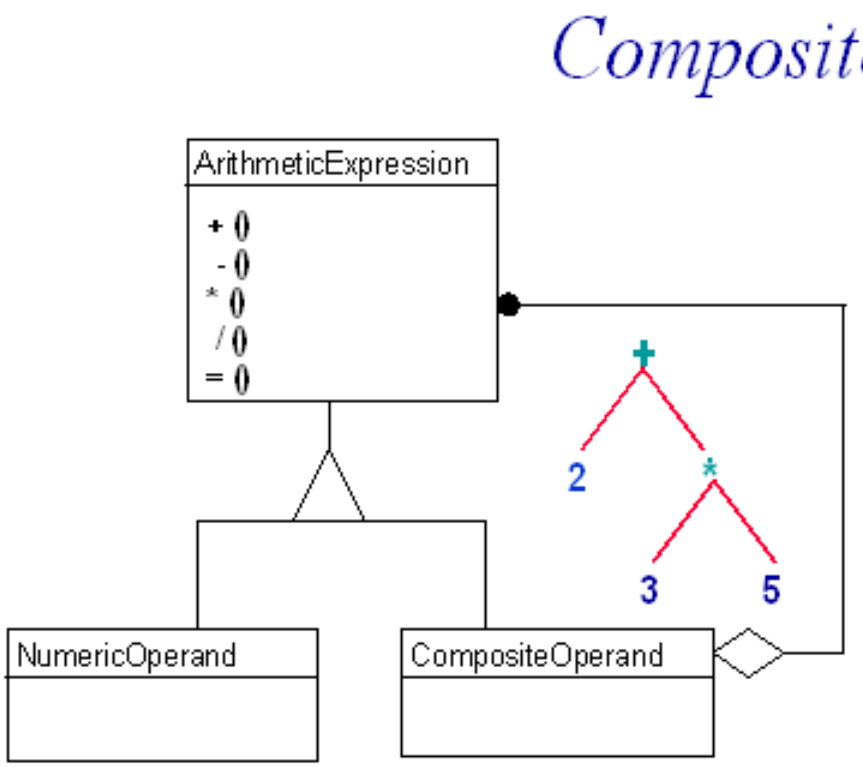


Composite

- Compose objects into tree structures that represent whole-part hierarchies. Composite lets clients treat individual objects and compositions of objects uniformlv. A leaf has the same interface as a node.



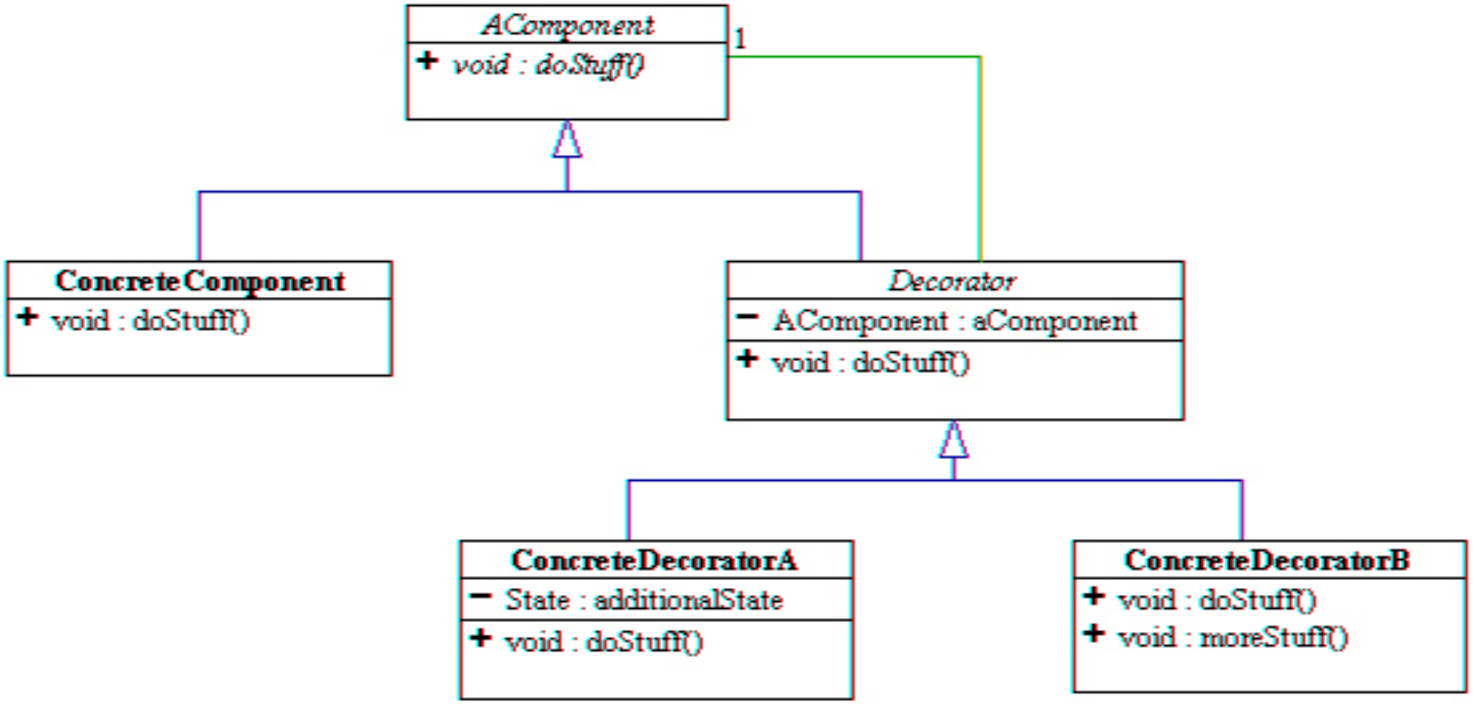
Composite sample



Arithmetic expressions can be expressed as trees where an operand can be a number or an arithmetic expression. Since the individual objects, and compositions of individual objects are treated uniformly, an arithmetic expression is an example of the *Composite* pattern.

Decorator

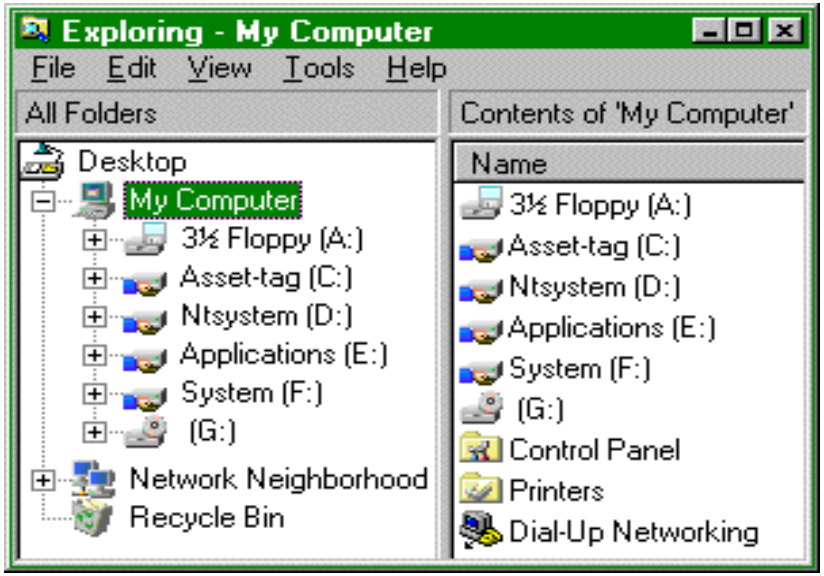
- Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality

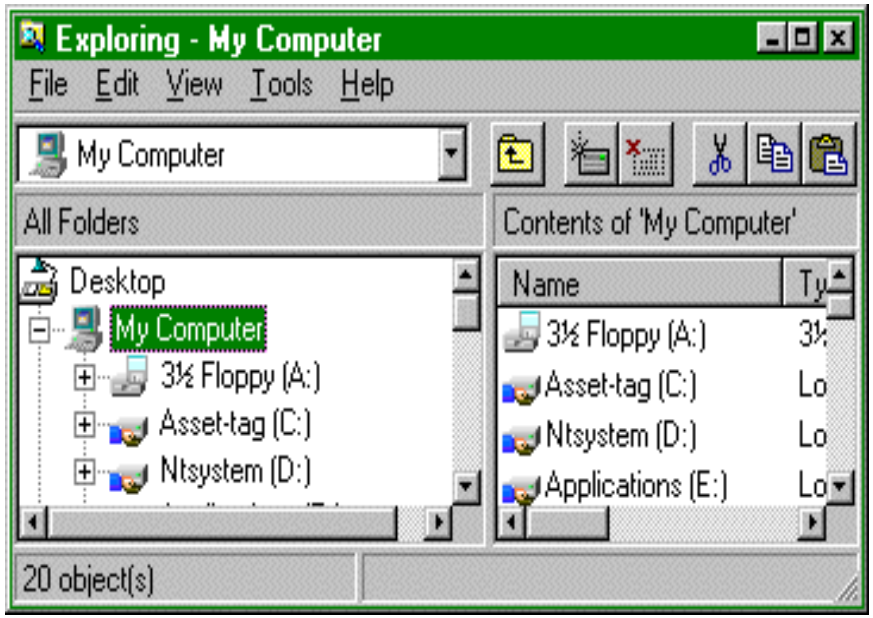


Decorator sample

```
FileOutputStream out = new FileOutputStream( "test.data" );
EncryptedOutputStream eos = new EncryptedOutputStream(out, key);
PrintStream pr = new PrintStream(eos);
pr.println("hola");
```

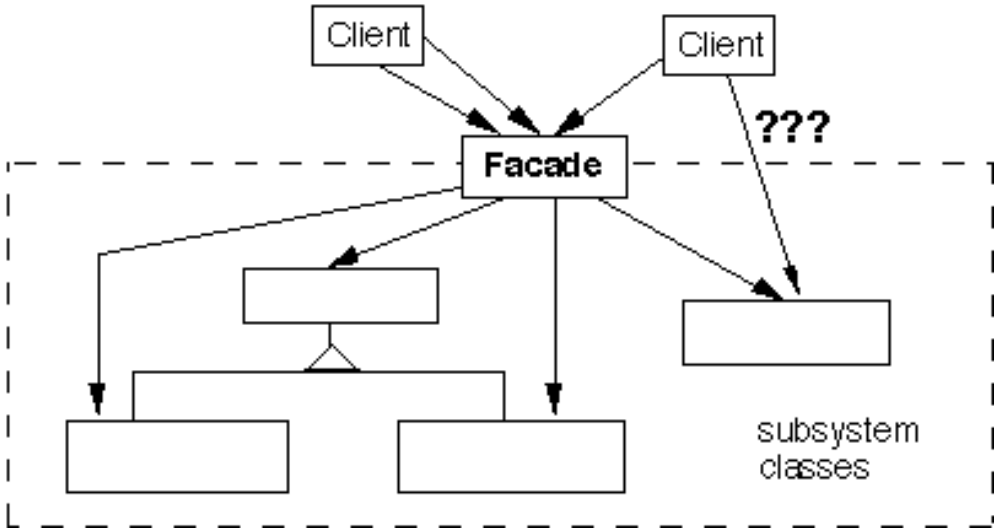
Another sample





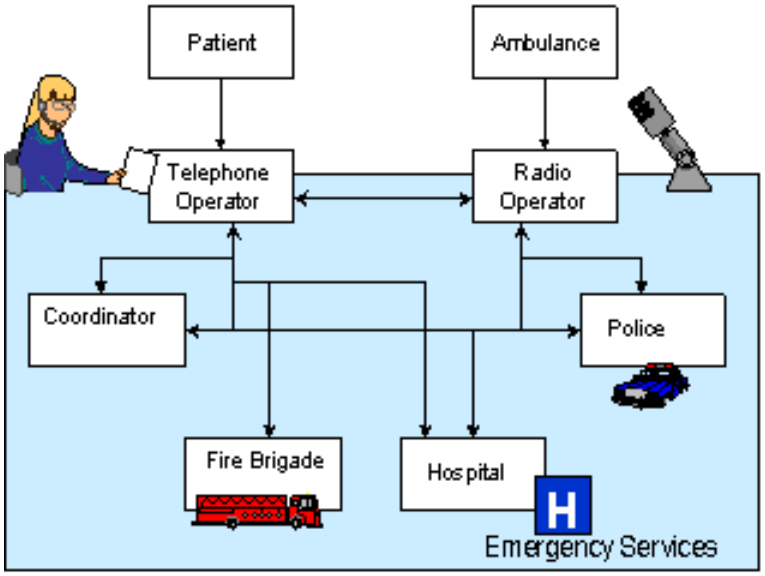
Facade

- Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use



Facade sample

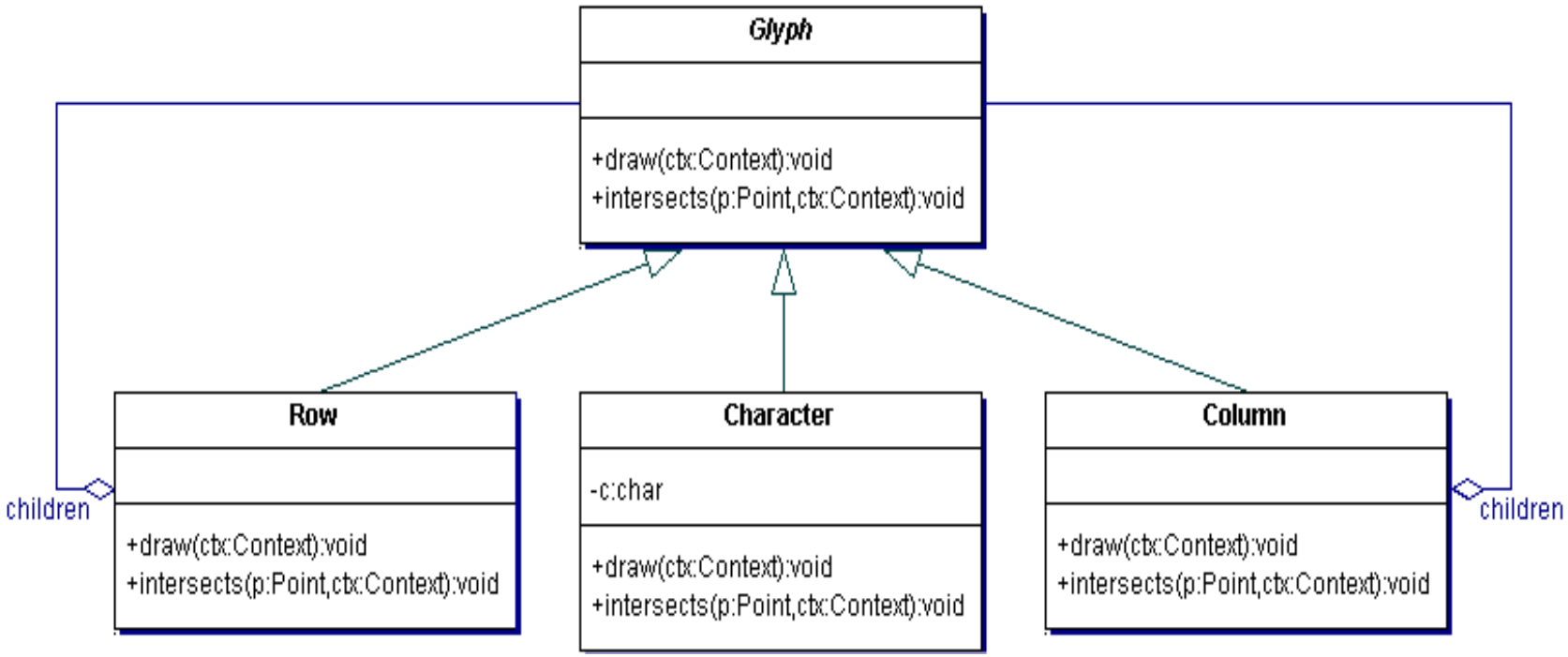
Facade



The headquarters of an ambulance service consists of several devices for different purposes, for instance ambulances with different levels of equipment and differently skilled crews, pediatric ambulances, helicopters, and so on. On the other hand easy use of this system – i.e.. calling for help – is essential.

Flyweight

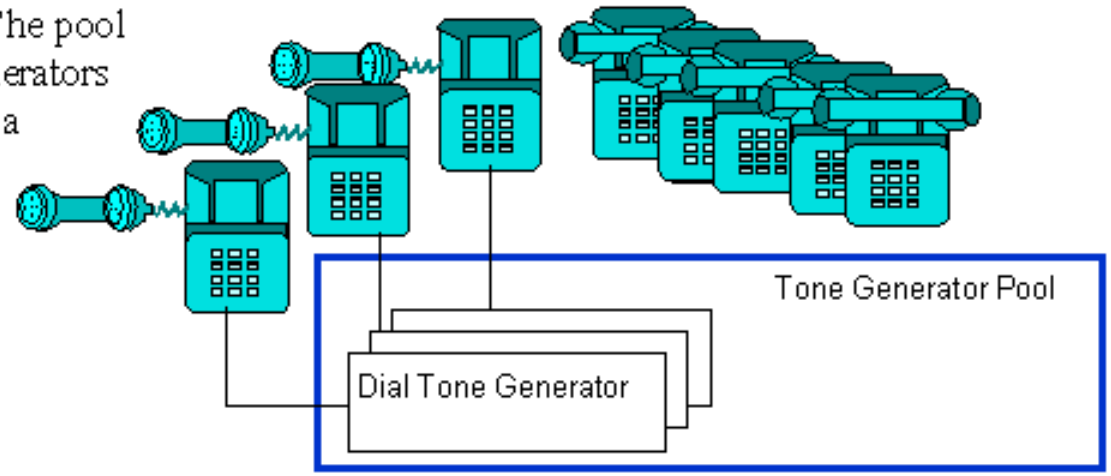
- Use sharing to support large numbers of fine-grained objects efficiently.



Flyweight sample

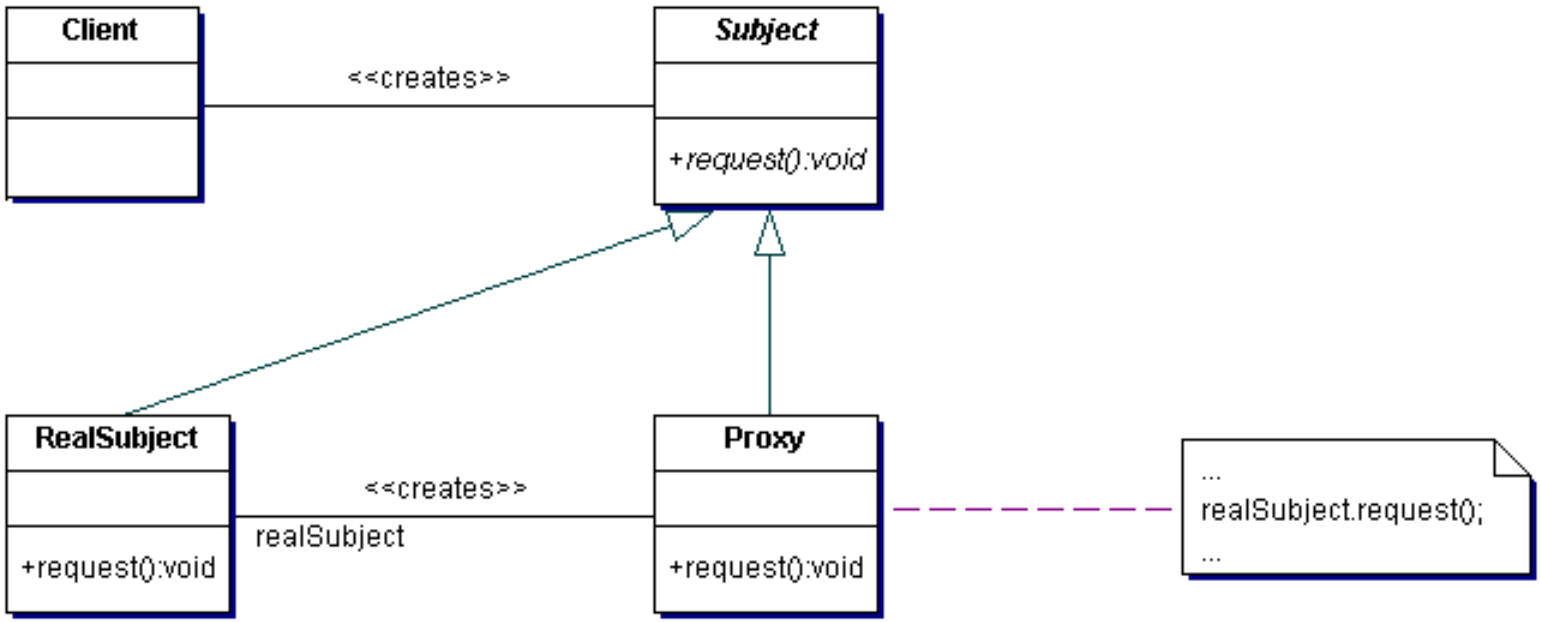
Flyweight

Most telephone subscribers are unaware that the pool of Tone generators (such as dial tone, busy tone or reorder tone) is much smaller than the number of subscribers. The pool of Dial Tone generators is an example of a *Flyweight*.



Proxy

- Provide a surrogate or placeholder for another object to control access to it.

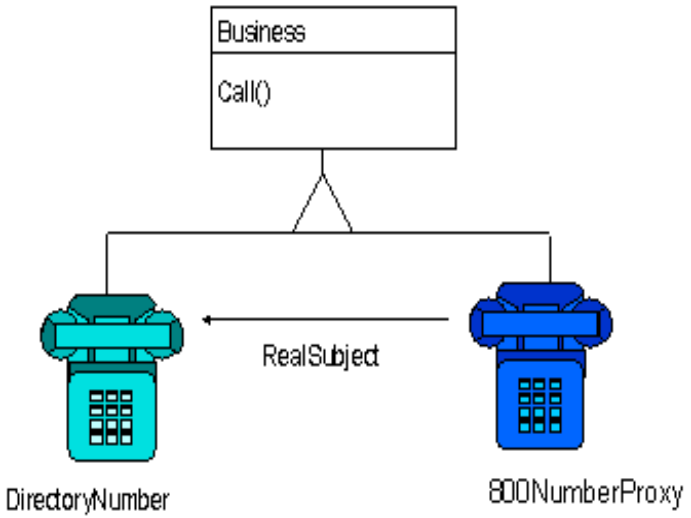


Proxy example

- Remote proxy
- Virtual proxy
- Security proxy
- Smart reference

Proxy

An 800 (8xx) number is a proxy for the real directory number. Callers will dial the 800 number, just as they would dial the actual directory number.

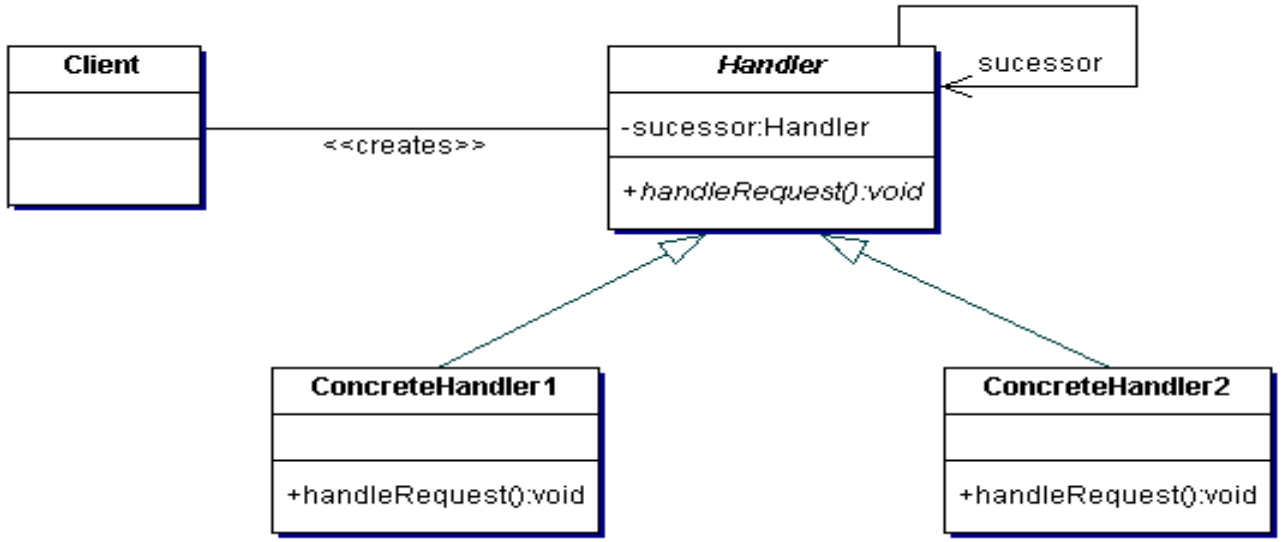


Behavioural patterns

- Behavioral patterns are concerned with algorithms and the assignment of responsabilities between objects.
- They describe not just patterns of objects or classes but also the patterns of communication between them.
- They shift focus away from flow of control and let you concentrate just on the way objects are interconnected.

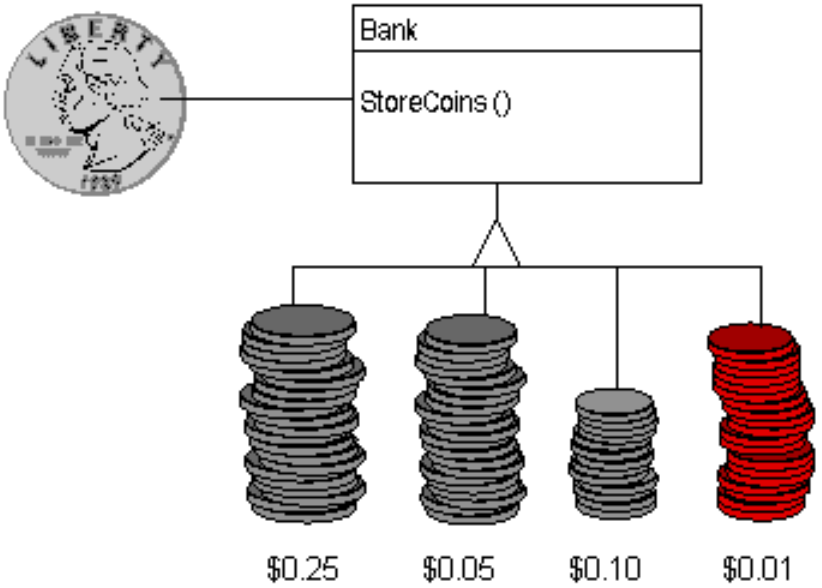
Chain of Responsability

- Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.



Example

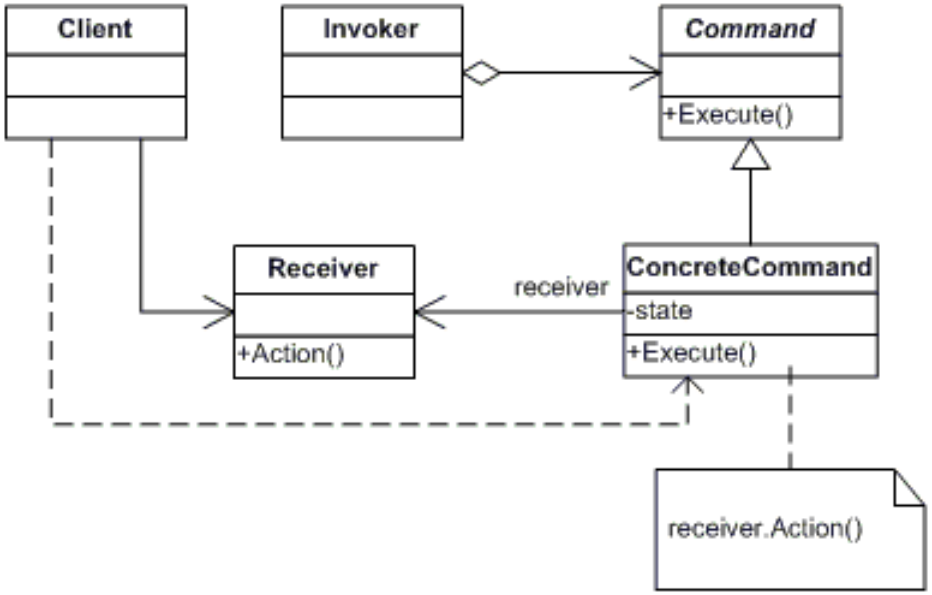
Chain of Responsibility



A mechanical sorting bank uses a single slot for all coins. As each coin is dropped, a *Chain of Responsibility* determines which tube accommodates each coin. If a tube cannot accommodate the coin, the coin is passed on until a tube can accept the coin.

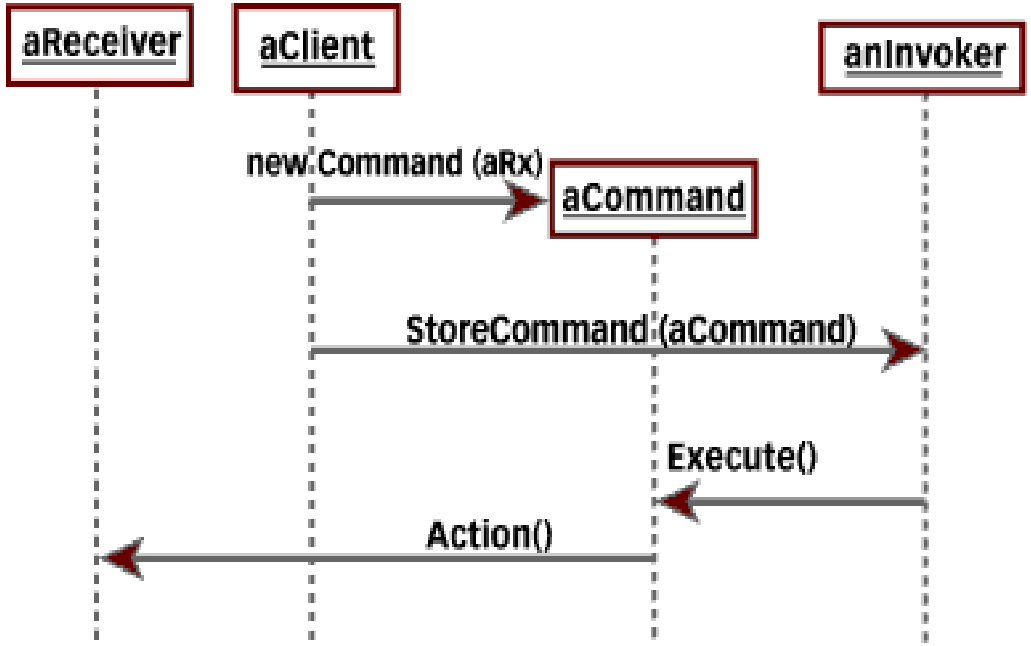
Command

- Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations



Command

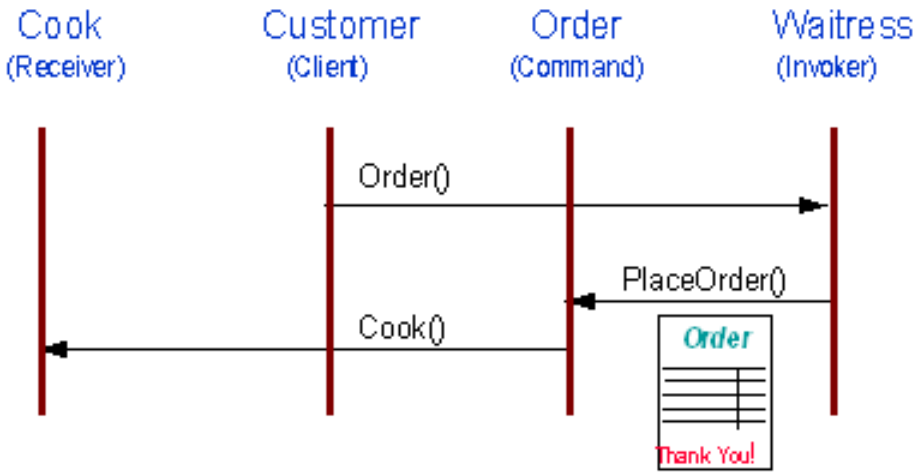
- The key idea here is that the concrete command registers itself with the Invoker and the Invoker calls it back, executing the command on the Receiver.



Command sample

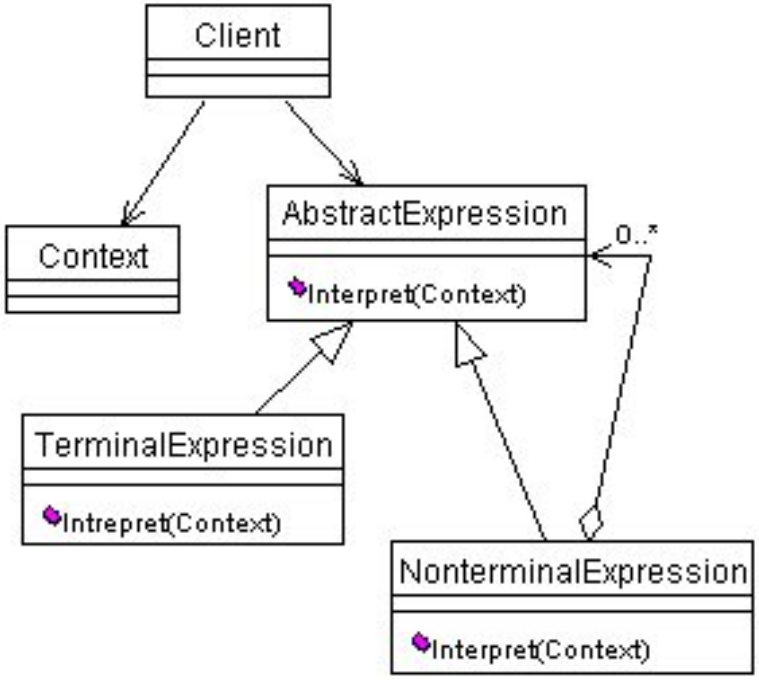
Command

The "check" at a restaurant is used to encapsulate the customer's order. The waitress takes the order, but it is the cook that carries out the *Command*. Since the pad of "checks" can be used by different restaurants, they can support many different commands

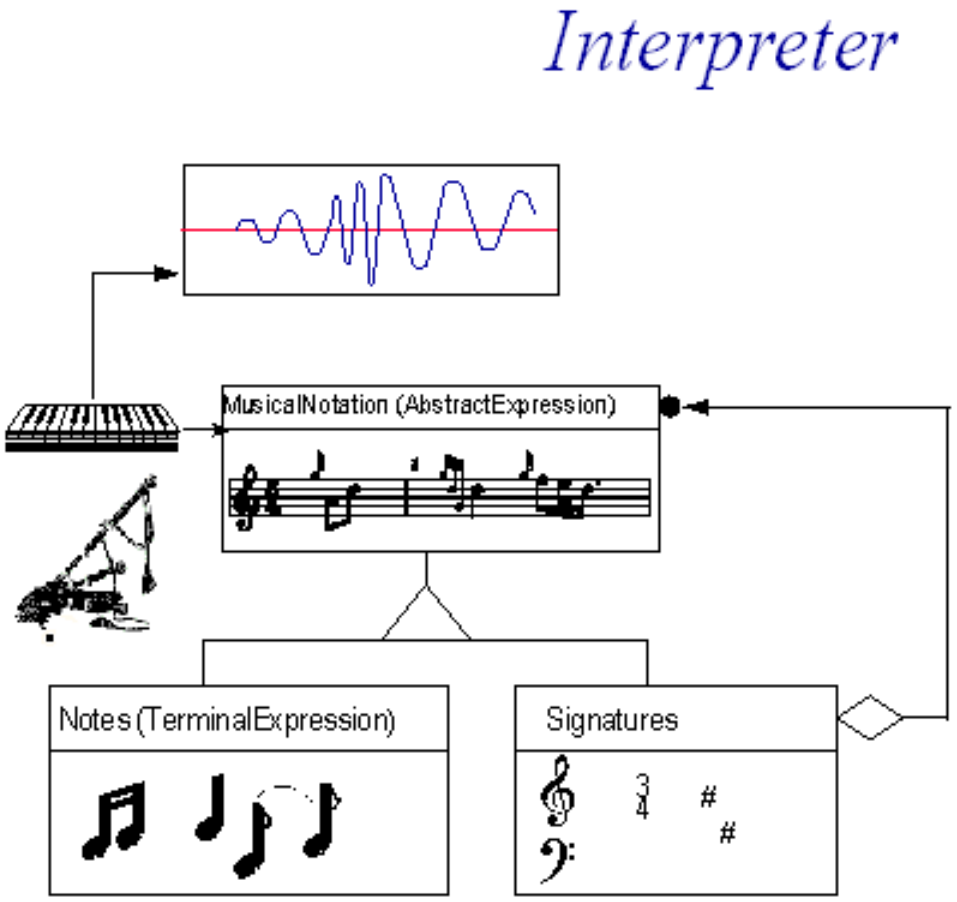


Interpreter

- Given a language, define a representation for its grammar along with an interpreter that uses the representation to interpret sentences in the language.



Interpreter sample

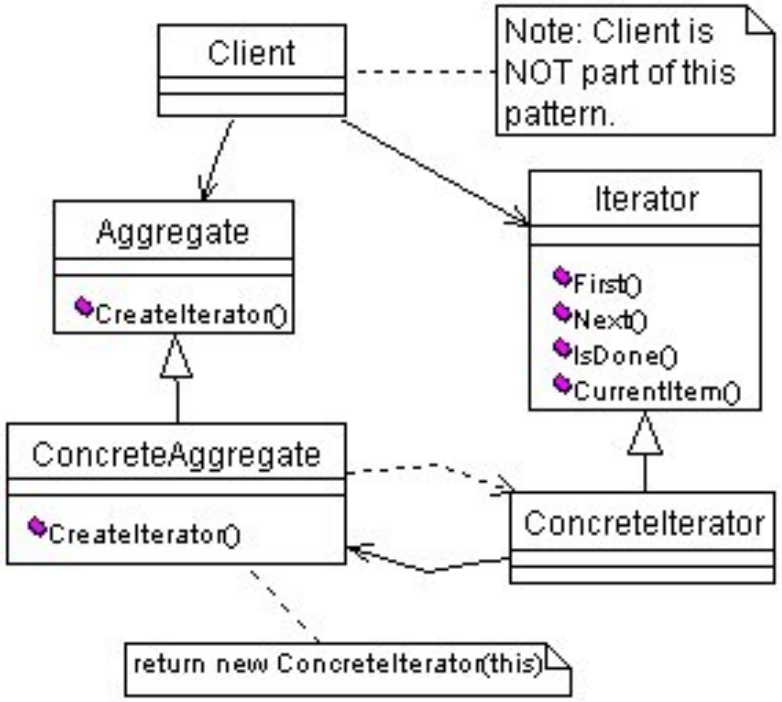


Musical notation provides a grammatical representation for the pitch and duration of sounds. When musicians play from a score, they are *Interpreters*, interpreting that grammar.

Behavioral

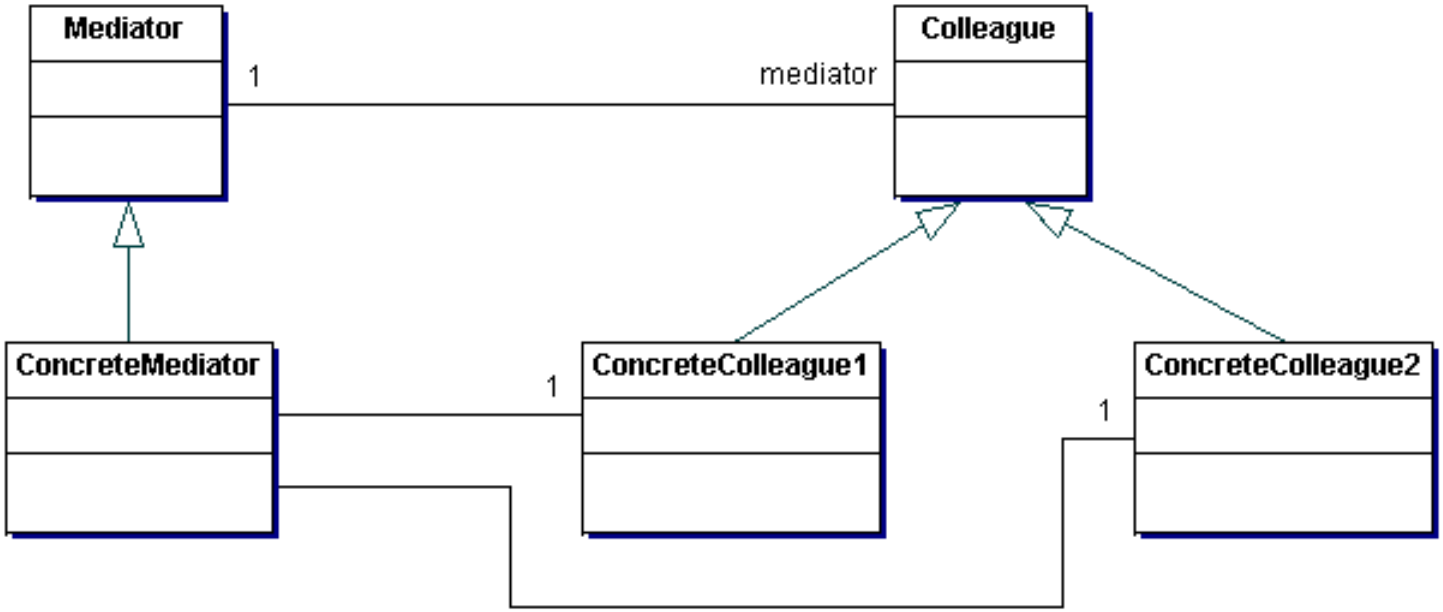
Iterator

- Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.



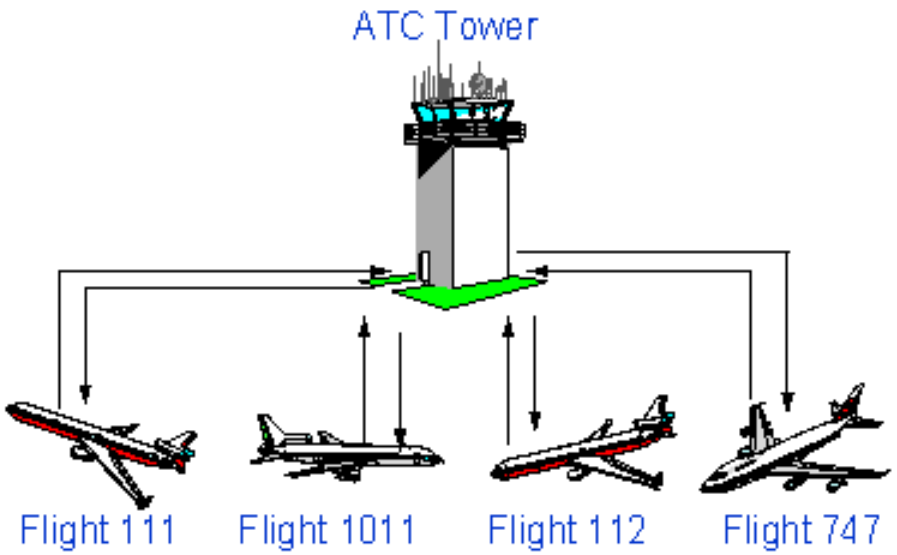
Mediator

- Define an object that encapsulates how a set of objects interact. Mediator promotes loose coupling by keeping objects from referring to each other explicitly, and it lets you vary their interaction independently.



Mediator sample

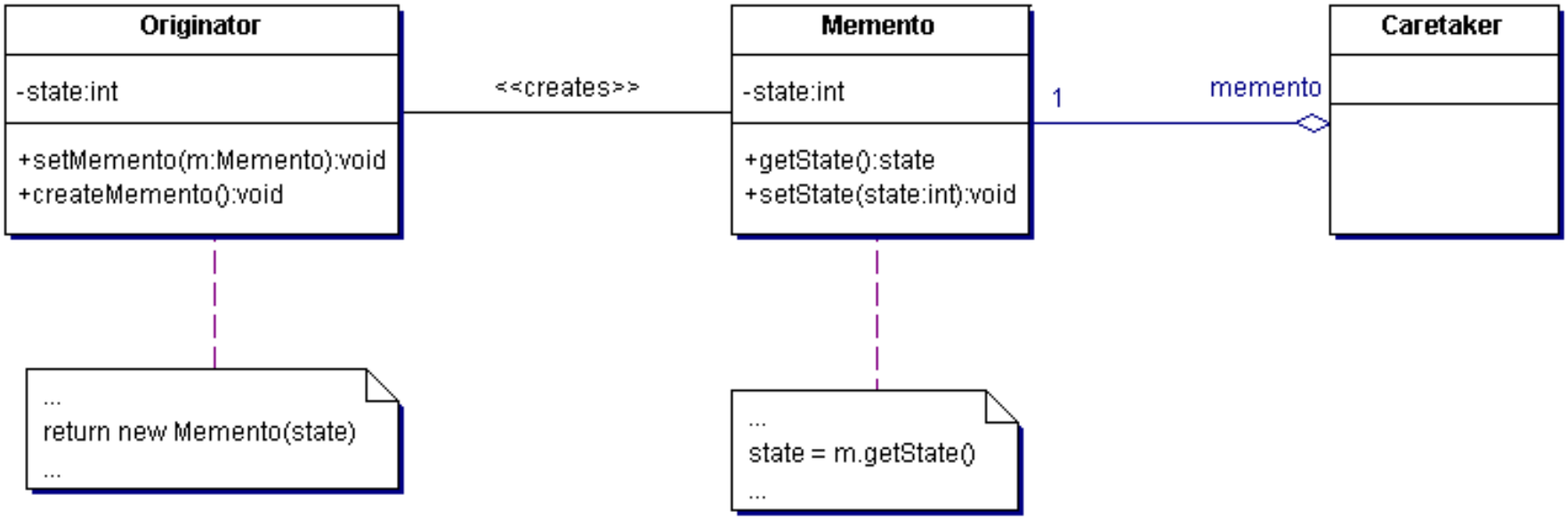
Mediator



The control tower at an airport provides a central point of communication for aircraft in the terminal area. Constraints on terminal area airspace are maintained by the tower. With the centralized communication and constraint maintenance, the tower behaves as a *Mediator*.

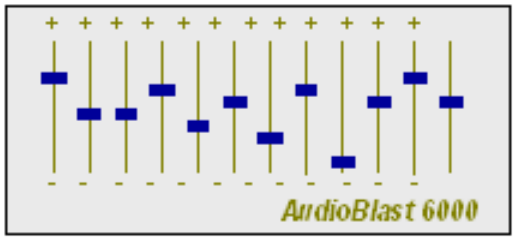
Memento

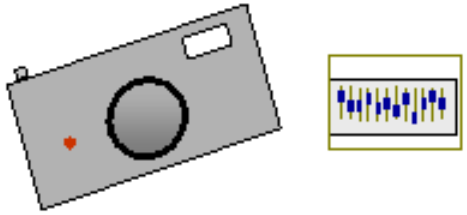
- Without violating encapsulation, capture and externalize an objects internal state so that the object can be restored to this state later.



Memento sample

Memento



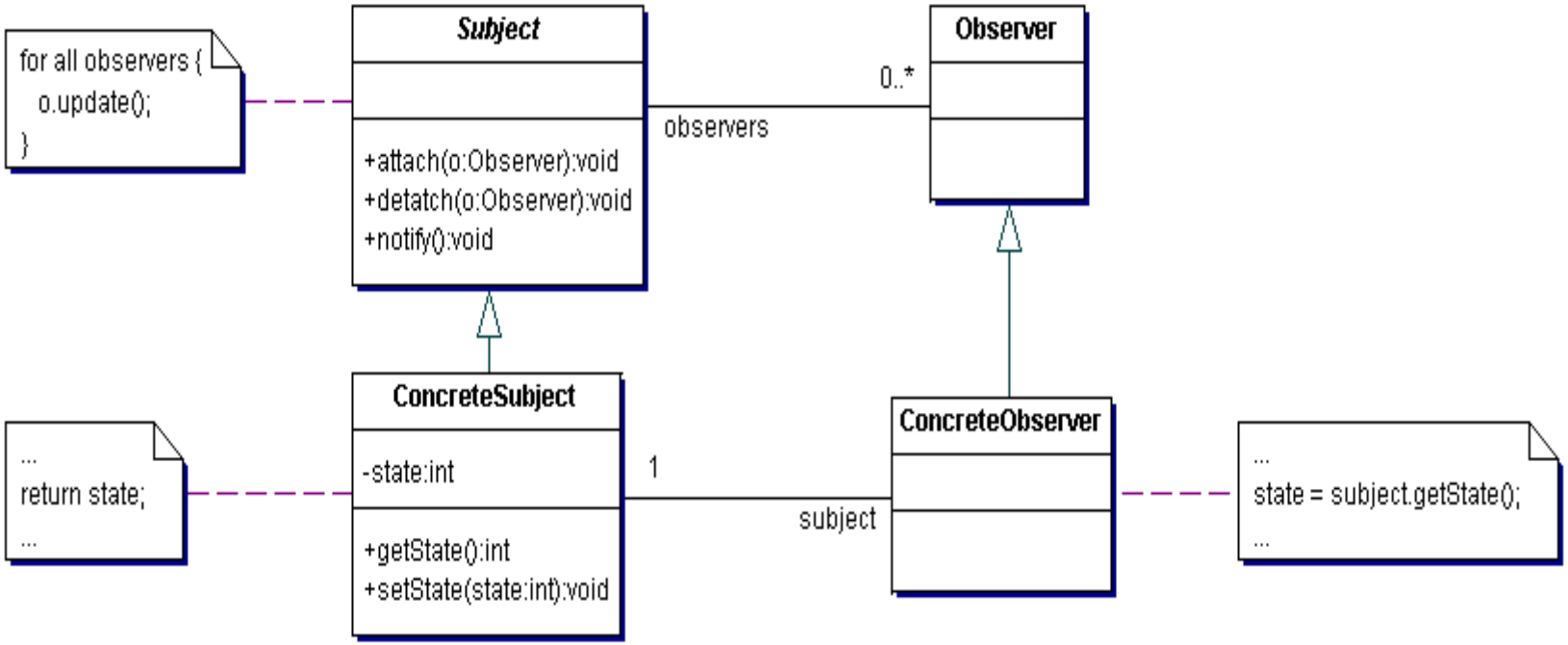


There can be an infinite number of settings for a piece of audio mixing equipment. An engineer could take a photograph of a particular setting, and use the photograph to restore the switch settings to the desired state if perturbed.

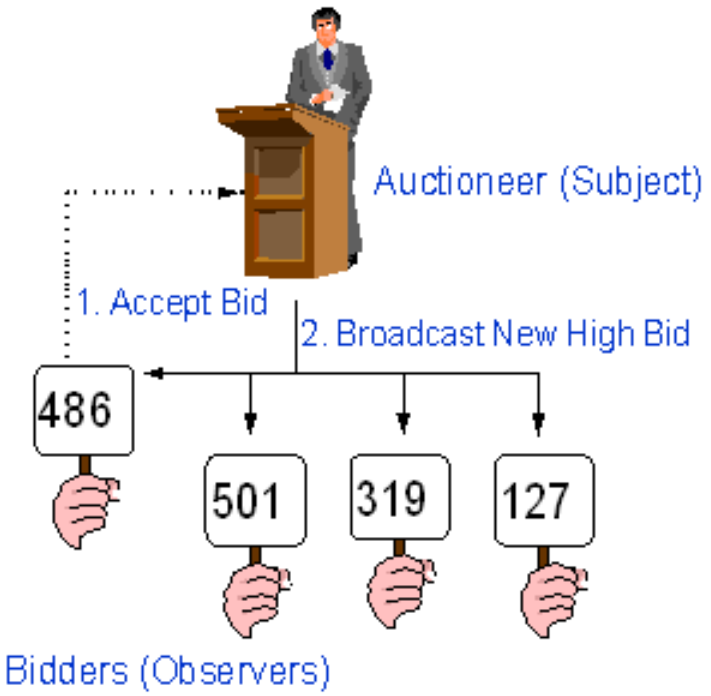
Behavioral

Observer

- Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.



Observer example



When a bidder at an auction accepts a bid, he or she raises a numbered paddle which identifies the bidder. The bid price then changes and all *Observers* must be notified of the change. The auctioneer then broadcasts the new bid to the bidders.

Java UI

- java.awt.event.ActionListener
  - void **actionPerformed**(ActionEvent e)
- java.awt.Button
  - void **addActionListener**(ActionListener l)

- ActionListener in the Observer pattern ?

```java
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class OnClick implements ActionListener {

    public void actionPerformed(ActionEvent arg0) {
        System.out.println("CLICK !!!!");

    }

}
```

```java
import java.awt.*;

public class ClickDemo {


    public static void main(String[] args) {
        Frame ventana = new Frame("ventana");
        Button boton = new Button("SI");
        ventana.add(boton);
        OnClick click = new OnClick();
        boton.addActionListener(click);
        ventana.pack();
        ventana.setVisible(true);

    }

}
```

Creating Events

```
import java.util.EventObject;

public class RemoteEvent extends EventObject
{

    private String eventType;
    private String content;

    public RemoteEvent (Object obj,String evType, String cont)  {
        super(obj);
        eventType = evType;
        content = cont;
    }

    public String getContent()  {
        return content;
    }

    public String getEventType()    {
        return eventType;
    }
}
```

Listener Interface

```
import java.util.EventListener;

public interface RemoteEventListener extends EventListener
{

    public void dataChanged(RemoteEvent evt);
}
```

```
import java.util.*;

public class PedroBean
{
    private String nombre = "";
    private Vector listeners = new Vector();

    public String getNombre()    {
        return nombre;
    }
    public void setNombre(String name)   {
        nombre = name;
        updateall();
    }
    public void updateall(){
        Iterator it = listeners.iterator();
        while (it.hasNext()){
            RemoteEventListener aListener = (RemoteEventListener)it.next();
            aListener.dataChanged(new RemoteEvent(this,"CHAT",nombre));
        }
    }
    public void addRemoteEventListener(RemoteEventListener remoter) {
            listeners.add(remoter);
        }
    public void removeRemoteEventListener (RemoteEventListener remoter) {
            listeners.remove(remoter);
    }
```

C# Events

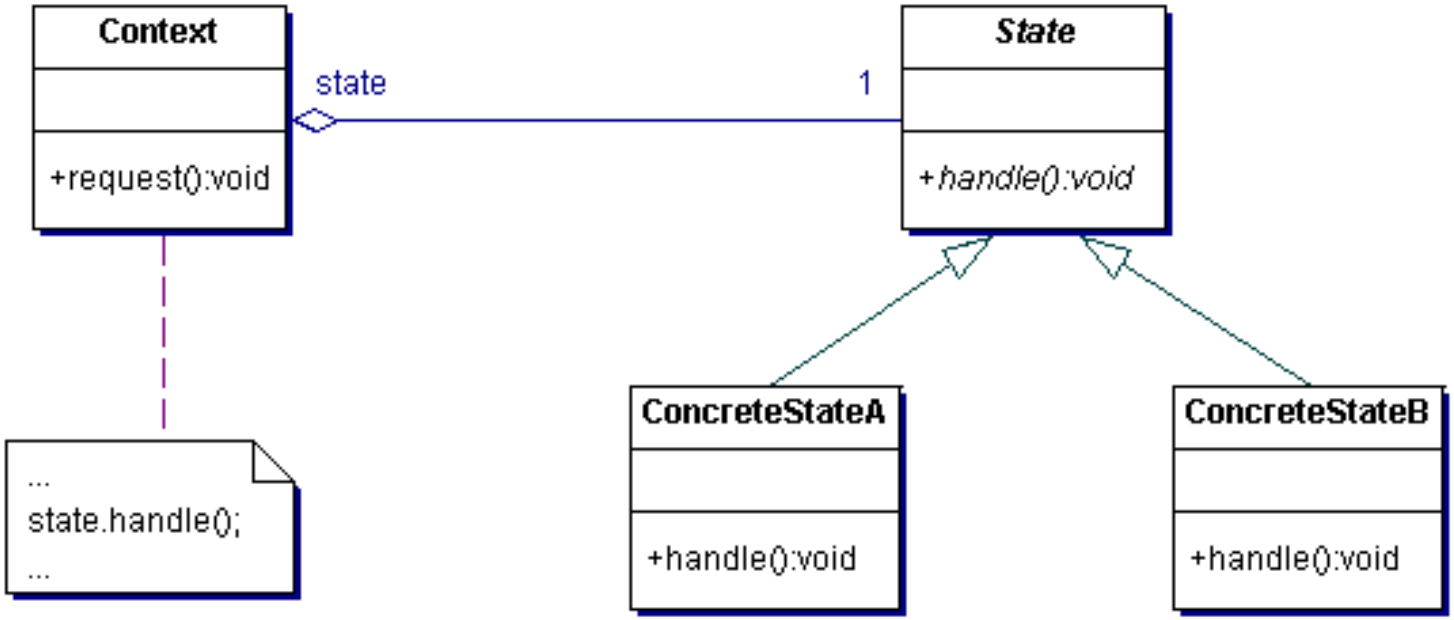
```
using System;
namespace wildert
{
    public class Metronome
    {
        public event TickHandler Tick;
        public EventArgs e = null;
        public delegate void TickHandler(Metronome m,
        EventArgs e);
        public void Start()
        {
            while (true)
            {
                System.Threading.Thread.Sleep(3000);
                if (Tick != null)
                {
                    Tick(this, e);
                }
            }
        }
    }
```

C# Events

```
public class Listener
        {
            public void Subscribe(Metronome m)
            {
                m.Tick += new
                      Metronome.TickHandler(HeardIt);
            }
            private void HeardIt(Metronome m, EventArgs e)
            {
                System.Console.WriteLine("HEARD IT");
            }
        }
    class Test
    {
        static void Main()
        {
            Metronome m = new Metronome();
            Listener l = new Listener();
            l.Subscribe(m);
            m.Start();
        }
```

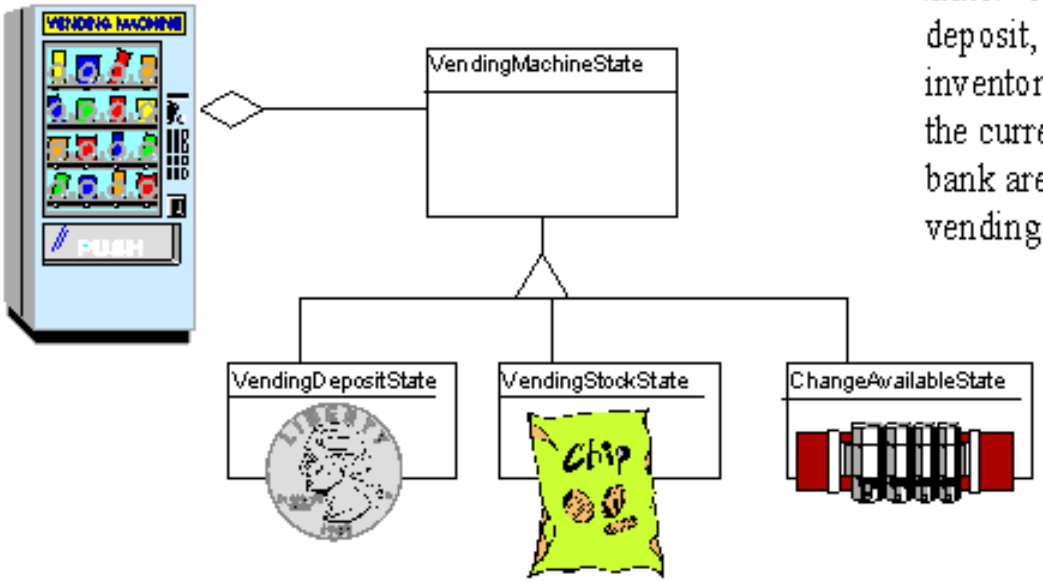
State

- Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.



State sample

State

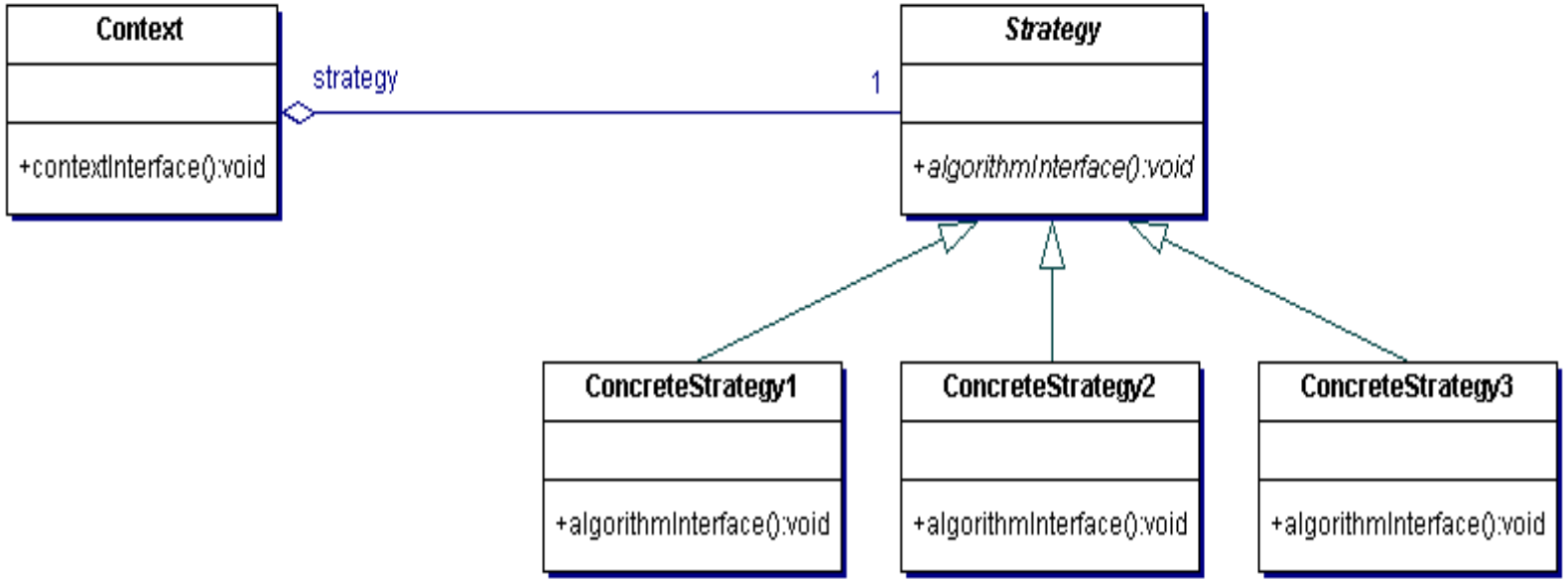


The behavior of a vending machine varies based on its *State*. The currency on deposit, the product inventory, the selection, and the currency in the change bank are all part of the vending machine's *State*.

Behavioral

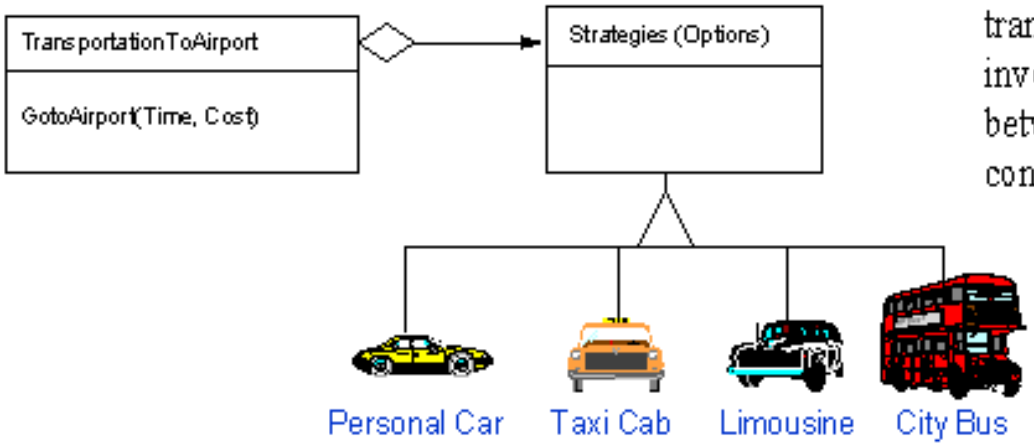
Strategy

- Define a family of algorithms, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from the clients that use it.



Strategy sample

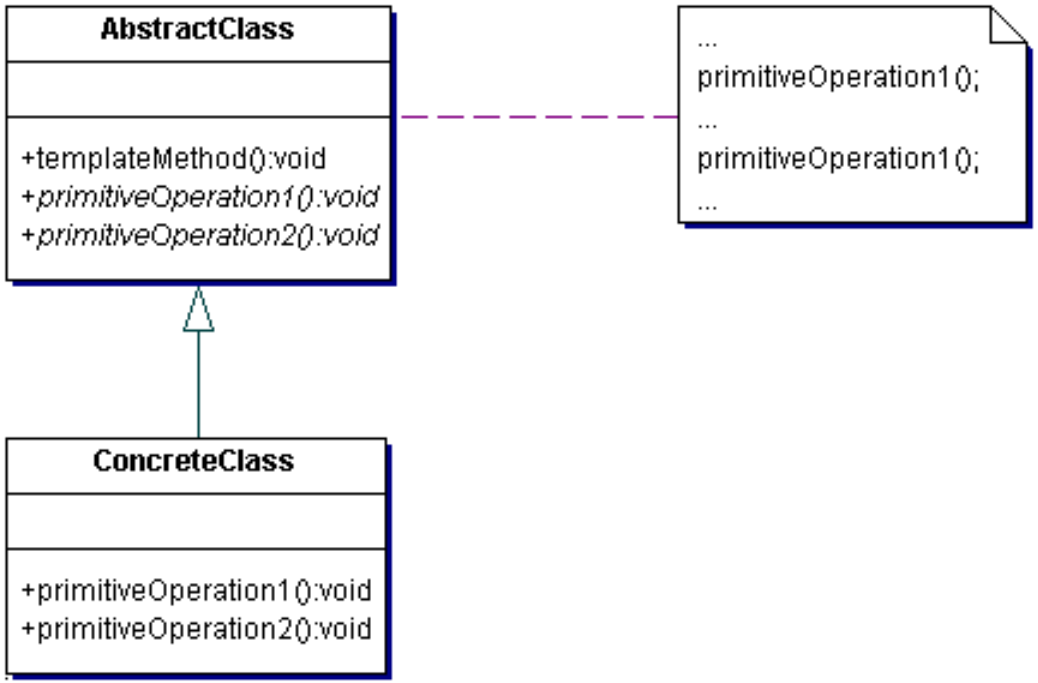
Strategy



There are many modes of transportation to and from an airport. Choosing a particular transportation *Strategy* involves making tradeoffs between cost, time, and convenience.

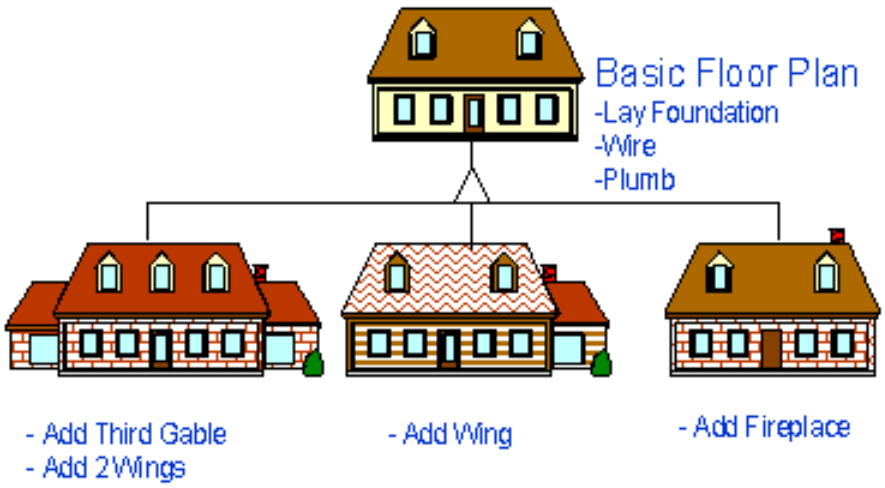
Template method

- Define the skeleton of an algorithm in an operation, deferring some steps to client subclasses. Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.



Sample

Template Method

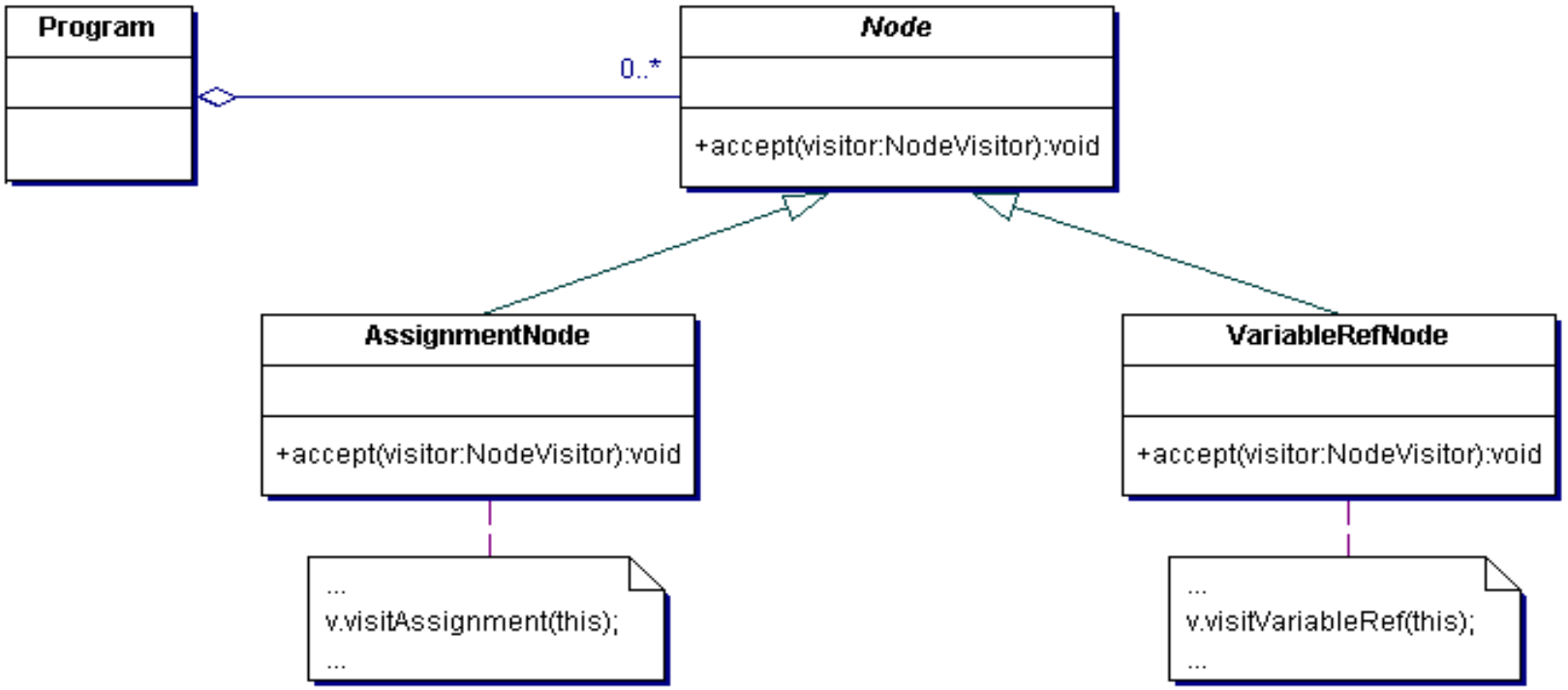


Variations added to Template Floor Plan

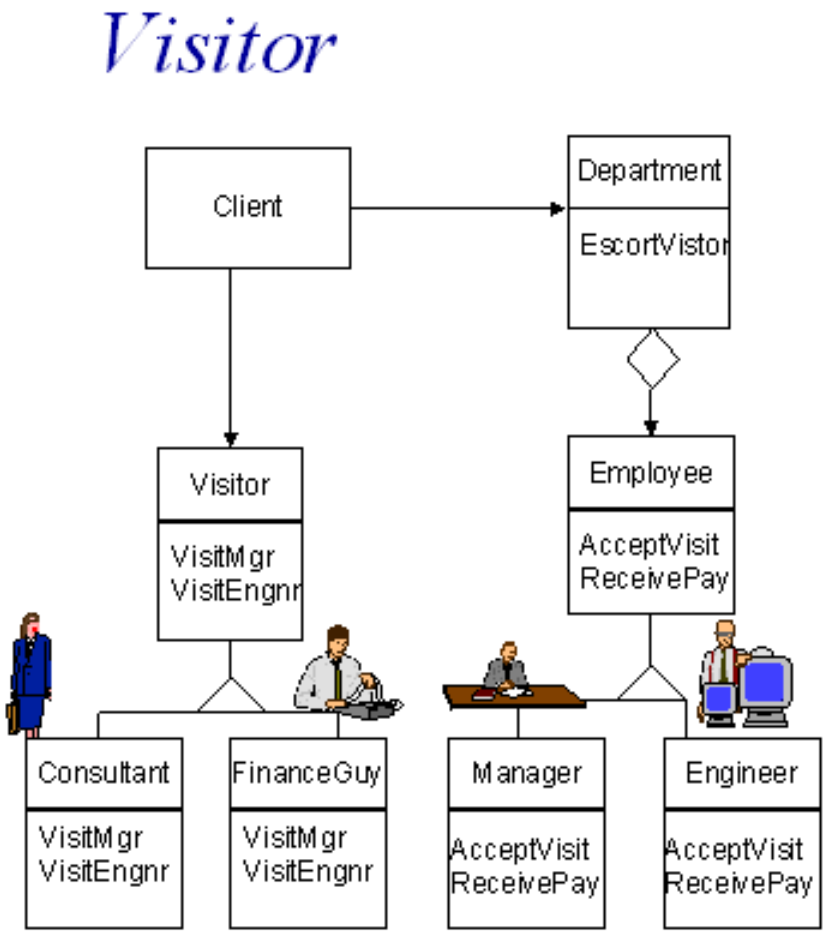
Subdivision developers often use a *Template Method* to produce a variety of home models from a limited number of floor plans. The basic floor plans are a skeleton, and the differentiation is deferred until later in the building process.

Visitor

- Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.

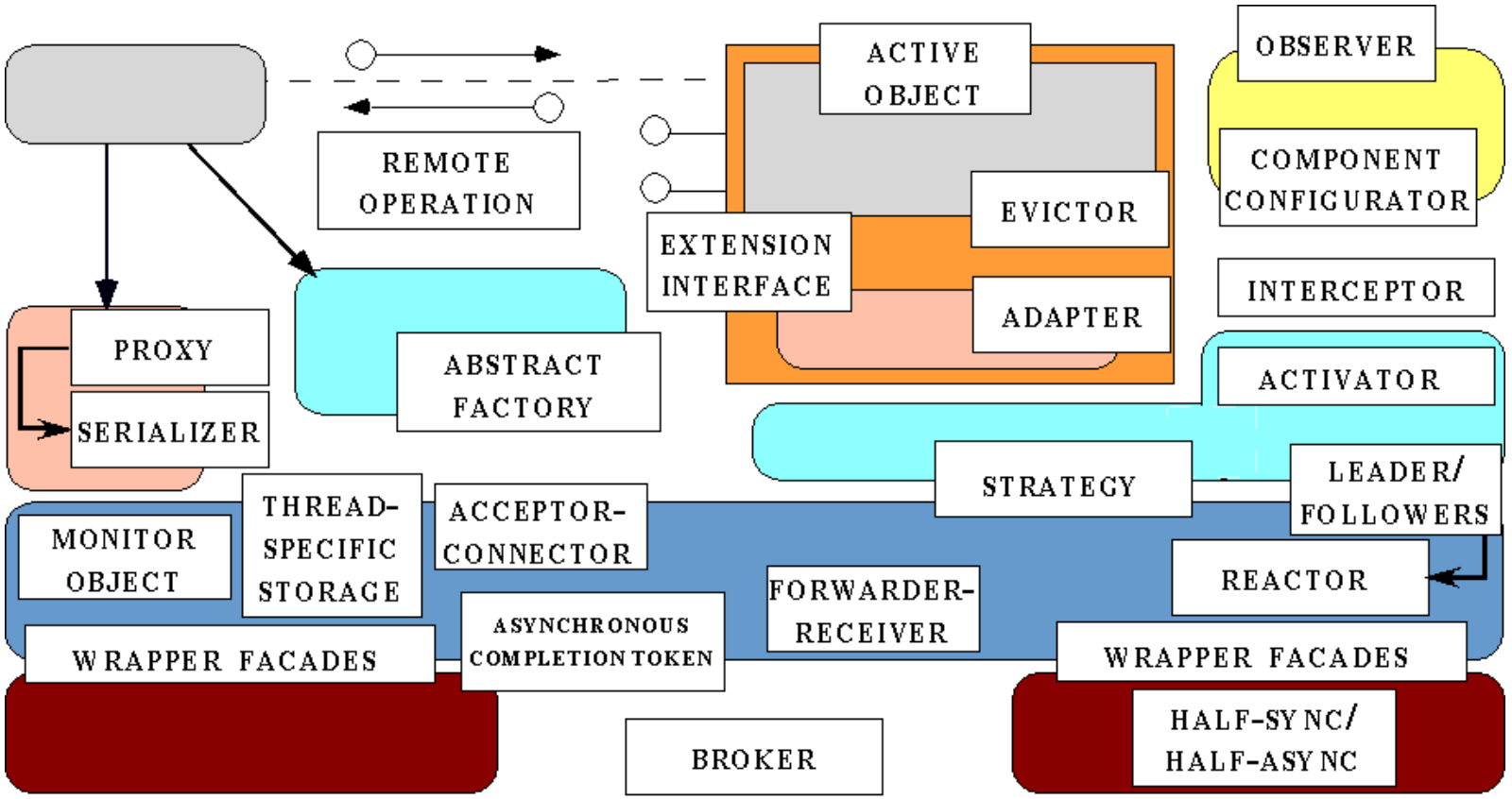


Visitor sample



An outside consultant coming into a department, to meet with everyone in the department on a one-on-one basis is an example of *Visitor*. While the visitor is escorted to each cubicle, she does nothing. Once she arrives at a cubicle, she springs into action interviewing the employee (sending messages and obtaining results).

POSA Patterns



POSA Patterns

- Layers
- Pipes and Filters
- Blackboard
- Broker
- MVC
- Presentation-Abstraction-Control
- Microkernel
- Reflection
- Whole-Part
- Master-Slave
- Proxy
- Command Processor
- View Handler
- Forwarder-Receiver
- Client-Dispatcher-Server
- Publisher-Subscriber
- Wrapper Facade
- Component Configurator
- Interceptor
- Extension Interface
- Reactor
- Proactor
- Asynchronous Completion Token
- Acceptor-Connector
- Scoped Locking
- Strageized Locking
- Thread-Safe Interface
- Double-Checked Locking Optimization
- Active Object
- Monitor Object
- Half-Sync/Half-Async
- Leader-Followers
- Thread-Specific Storage

Conclusions

- Patterns = Reuse
- Patterns promote loose coupling using delegation
- Good and elegant software use well-known patterns
- Micro-architecture elements
- Study and use patterns !!!!